# Backend mövzularını addım addım başa düşək

Bir kafe tətbiqi üzərindən doqquz mövzunu öyrənəcəyik. Müştəri menyunu açır, sifariş verir, server məlumatı saxlayır və cavab qaytarır. Bütün dərslər bu tanış hadisə ətrafında qurulub.

Hər hissə əvvəl problemi göstərir, sonra həllini izah edir. Yeni söz həmin problemə lazım olanda açılır. Kod nümunəsinin ardınca onun nə etdiyi və hansı nəticəni verdiyi göstərilir.

**Oxu qaydası:** əvvəl nümunəni başa düş, sonra qalın yazılmış nəticəni öz sözlərinlə de. Sonda verilən kiçik sualı cavablandır. Cavab alınmırsa bütün kitabı yenidən oxumaq əvəzinə həmin nümunəyə qayıt.

Kodlar öyrətmək üçün kiçik saxlanıb. Fraqment adlandırılan hissə mövcud layihənin müəyyən yerinə əlavə olunur. Göstərilməyən sinif və parametrlərin nə olduğu mətnin içində açıqlanır.

DI və EF Core loading dərsləri 31 oktyabr 2026 tarixində genişləndirilib.

# Dərslərin ardıcıllığı

1 Sorğu serverdə necə işləyir . . . . . 3

2 DI asılılığı kənardan verməkdir . . . . . 8

3 Deferred execution, IEnumerable, IQueryable və loading . . . . . 43

4 Function və stored procedure nə ilə fərqlənir . . . . . 63

5 İndeks məlumatı tapmağı necə dəyişir . . . . . 70

6 Fraqmentasiya məlumatın yayılmasıdır . . . . . 81

7 SOLID dəyişiklikləri nəzarətdə saxlamaq üçündür . . . . . 88

8 Pattern təkrarlanan problem üçün həll quruluşudur . . . . . 95

9 Async gözləyərkən threadi boş saxlamağa kömək edir . . . . . 108

# 1 Sorğu serverdə necə işləyir

Kafedə müştəri telefondan menyunu açır. Ekranda məhsulların görünməsi üçün telefon serverdən siyahı istəyir. **Server** burada məlumatı hazırlayıb cavab verən proqramdır.

Bu dərsdə yalnız bir hadisəni izləyəcəyik: müştəri menyunu istəyir, server menyunu qaytarır. Həmin hadisənin başlanğıcdan sona qədər keçdiyi yol onun **həyat dövrüdür**. İngiliscə buna **lifecycle** deyilir.

Müştəri ilə server arasındakı yol

```
Telefon: "Menyunu ver."
             |
             v
Server: menyunu tapir ve cavab hazirlayir.
             |
             v
Telefon: gelen menyunu ekranda gosterir.
```

Telefonun göndərdiyi istək **request**, serverin qaytardığı cavab **response** adlanır. “Sorğu” və “cavab” deyəndə bunları nəzərdə tutacağıq.

**Bu mərhələdə yadda saxla:** server bir sorğunu qəbul edir, onun üçün iş görür və cavab qaytarır. Lifecycle həmin işin yolunu təsvir edir.

# Sorğunun içində nə gəlir

Telefon serverə sadəcə “menyu” sözünü göndərmir. Sorğunun hansı işi istədiyini və hansı ünvana getdiyini bildirir. Məsələn:

HTTP sorğusunun sadələşdirilmiş görünüşü

```
GET /menu
```

**HTTP** müştəri ilə serverin sorğu və cavabı hansı qayda ilə göndərdiyini müəyyən edir. Yuxarıdakı `GET` məlumat oxumaq istədiyimizi bildirir. `/menu` hansı məlumatı istədiyimizi göstərən yoldur.

Serverdə `/menu` üçün yazılmış kod seçilir. Ünvanla onu işləyəcək kodun uyğunlaşdırılmasına **routing** deyilir. Seçilən işləyici nöqtəyə isə **endpoint** deyilir.

Server menyunu tapanda `200` statusu ilə cavab verə bilər. **Status kodu** cavabın nəticəsini bildirən rəqəmdir. Menyu tapılmırsa `404`, gözlənilməz server xətası varsa `500` ola bilər.

Beləliklə, ardıcıllıq budur: **ünvan gəlir, həmin ünvanın kodu seçilir, kod cavab hazırlayır.**

İlk oxunuşda yadda saxlamalı əsas əlaqə budur: `GET /menu` sorğudur; həmin sorğuya cavab verən kod endpoint-dir.

# Sorğu niyə birbaşa menyu koduna getmir

Kafedə bəzi əməliyyatlar yalnız daxil olmuş istifadəçilər üçündür. Məsələn, “mənim sifarişlərim” ekranını açmazdan əvvəl istifadəçini tanımaq lazımdır.

Server buna görə əsas işi görməzdən əvvəl bir neçə yoxlamadan keçir. Bu yoxlamaları aparan komponentlərə **middleware** deyilir. Onların ardıcıllığı **pipeline**, yəni emal zənciridir.

Bir sorğunun nümunə yolu

```
Sorqu gelir
    |+    v
Xeta yaranarsa onu tutacaq qat
    |
    v
Istifadecinin kim oldugunun yoxlanmasi
    |
    v
Istifadecinin bu ise icazesinin yoxlanmasi
    |
    v
Sifarisleri getiren esas kod
```

Kimliyi müəyyən etməyə **authentication** deyilir: “Bu adam kimdir?” İcazəni yoxlamağa **authorization** deyilir: “Bu adam bu işi görə bilərmi?” Daxil olmuş müştərinin başqa müştərinin sifarişinə baxmağa icazəsi olmaya bilər.

Middleware sorğunu növbəti mərhələyə ötürə və ya özü cavab verib dayandıra bilər. İcazə yoxdursa əsas sifariş koduna getmədən cavab qaytarılır.

**Nəticə:** middleware əsas işə çatana qədər və iş bitəndən sonra görüləcək ortaq işləri təşkil edir.

# Əvvəl və sonra kodunun sırası

İki middleware düşün. Birincinin adı A, ikincinin adı B-dir. Hər ikisi əvvəl bir qeyd yazır, növbəti mərhələni çağırır, sonra yenə qeyd yazır.

İşləmə sırası

```
A: evvel
    B: evvel
        Menyunu hazirlayan kod
    B: sonra
A: sonra
```

Sıra niyə belədir? A, B-ni çağırıb onun bitməsini gözləyir. B əsas kodu çağırıb onun bitməsini gözləyir. Əsas kod bitəndə əvvəl B-nin, sonra A-nın qalan hissəsi işləyir.

ASP.NET Core-da növbəti mərhələni təmsil edən parametr çox vaxt `next` adlanır. **ASP.NET Core** C# ilə veb server proqramı qurmaq üçün istifadə edilən çərçivədir. Çərçivə hazır server və sorğu emalı mexanizmlərini təqdim edir.

Əgər B növbəti mərhələni çağırmasa, əsas kod işləməz. Əgər əsas kod xəta ilə dayansa, “sonra” hissələrinin hamısının adi qaydada işləyəcəyinə güvənmək olmaz. Xətadan asılı olmayaraq görülməli təmizlik işi üçün `finally` bloku istifadə edilir. Bu blokun işi koddan çıxarkən təmizliyi yerinə yetirməkdir.

**Yadda saxla:** sorğu zəncirin içinə doğru gedir, uğurlu iş bitəndə davam kodu əks ardıcıllıqla tamamlanır.

# Sorğu bitəndə nə bitir

Telefon menyunu aldı. Bu sorğu üçün iş bitdi. Amma server proqramı bağlanmır: növbəti müştərinin sorğusunu qəbul etməyə davam edir.

Burada iki müxtəlif ömür var:

| Nəyin ömrü | Nə vaxt bitir |
|---|---|
| Bir HTTP sorğusu | Həmin sorğunun emalı tamamlananda. |
| Server proqramı | Tətbiq dayandırılanda və ya proses bağlananda. |

**Proses** əməliyyat sistemində işləyən proqramın mühitidir. Eyni server proqramı çoxlu sorğunu emal edə bilər.

Sorğu üçün yaradılmış köməkçi obyektlərin də öz ömrü olur. Bəzisi yalnız bu sorğu üçün, bəzisi isə bütün server üçün paylaşılır. Bunun necə seçildiyini növbəti DI dərsində görəcəyik.

Müştəri əlaqəni kəsəndə server bunu ləğv siqnalı ilə bildirə bilər. Amma başladılmış hər iş özbaşına dərhal yox olmur. İşi görən kod siqnalı nəzərə almalıdır. Bunu async dərsində ayrıca açacağıq.

### Özünü yoxla

**Telefon eyni menyu ekranını iki dəfə yeniləsə nə olur?** Adətən iki ayrı HTTP sorğusu göndərilir. Server proqramı isə həmin iki sorğu arasında işləməyə davam edir.

**İcazə yoxlaması əsas kodu çağırmadan cavab verə bilərmi?** Bəli. Middleware zənciri orada dayandıra bilər.

# 2 DI asılılığı kənardan verməkdir

Sifariş tamamlananda qəbz hazırlayan kiçik proqram quracağıq. Əvvəl işləyən DI-sız kodu, sonra eyni işi DI ilə görən kodu oxuyacağıq. Nəticə dəyişməyəcək; obyektlərin harada yaradıldığı və necə birləşdirildiyi dəyişəcək.

**DI - Dependency Injection** bir obyektin ehtiyac duyduğu başqa obyektin ona kənardan verilməsidir. Dependency asılılıq, injection isə həmin asılılığın ötürülməsi deməkdir.

Bu dərsdə anlayışları aşağıdakı sıra ilə açırıq:

- Əvvəl DI-sız və DI ilə kod, sonra nəzəri fərq.
- Interface ilə konkret sinfə bağlılığın azaldılması.
- Container-in gördüyü iş.
- Sorğu və scope: bir işin başlanğıc və bitiş sərhədi.
- Resolve: container-dən xidmət əldə etmək.
- Transient, scoped və singleton: yeni və ya eyni obyektin verilməsi.
- İşlədilə bilən Console və veb nümunələri.
- Injection üsulları, lifetime səhvləri və resursların təmizlənməsi.

İlk oxunuşda real veb nümunəsinin nəticələrinə qədər ardıcıl get. Sonrakı səhifələr artıq öyrəndiyin məntiqi genişləndirir. Əlavə terminlər əsas nümunəyə ehtiyac yarandıqda təqdim edilir.

Kodlarda qəbz hazırlamaq sadəcə mətn qaytarmaqdır. Printer cihazına qoşulmur, e-poçt göndərilmir, PDF yaradılmır. Beləliklə, diqqət yalnız obyektlər arasındakı əlaqədə qalır.

**Bu dərsin əsas sualı:** lazım olan obyekti kim yaradır, kim verir və kim istifadə edir?

# DI-sız proqram: printer-i sifariş sinfi özü yaradır

Bu, Console layihəsinin tam `Program.cs` nümunəsidir. Sonrakı alternativ nümunələrdə həmin fayl əvəz edilir; eyni adlı sinifləri bir fayla təkrar əlavə etmirik.

C# - Program.cs, DI-sız variant

```
var orders = new OrderService();
Console.WriteLine(orders.Complete());

public class Printer
{
    public string Print()
    {
        return "Qebz hazirdir";
    }
}

public class OrderService
{
    private Printer _printer = new Printer();

    public string Complete()
    {
        return _printer.Print();
    }
}
```

Ekrandakı nəticə

```
Qebz hazirdir
```

`OrderService` yarananda onun `_printer` sahəsi üçün də `new Printer()` işləyir. Sifariş obyekti öz asılılığını özü yaradır. Bu işləyən koddur; məsələ onun dəyişikliklərə necə uyğunlaşacağıdır.

# DI-sız kodda hər sətrin mənası

**Sinif - class** obyektin quruluşunu və edə bildiyi əməliyyatları müəyyən edir. `Printer` və `OrderService` sinif adlarıdır. **Obyekt - instance** həmin sinifdən yaradılan konkret nüsxədir.

`new OrderService()` sifariş obyektini yaradır. `var orders` həmin obyektə istinadı saxlayan dəyişəndir. `var` dəyişənin tipinin sağdakı ifadədən müəyyən edilməsinə imkan verir.

Yaradılma və istifadə ardıcıllığı

```
1. new OrderService() isleyir.
2. Onun sahesi ucun new Printer() isleyir.
3. orders.Complete() cagirilir.
4. Complete daxilinde _printer.Print() cagirilir.
5. Metn neticesi ekrana yazilir.
```

**Sahə - field** obyektin daxilində saxlanan məlumatdır. `_printer` belə sahədir. _ işarəsi sadəcə adlandırma seçimidir. `private` sahənin yalnız həmin sinfin daxilində əlçatan olduğunu bildirir.

**Metod** çağırılan əməliyyatdır. `Complete` və `Print` metod adlarıdır. `string` mətn tipidir. `return` nəticəni çağıran koda verir. `Console.WriteLine` onu ekrana yazır. `public` hissənin kənardan əlçatan olduğunu göstərir.

`OrderService`-in işləməsi `Printer` obyektinə bağlıdır. Buna görə Printer onun **asılılığıdır**. Hazırkı kod həm sifarişi tamamlayır, həm də hansı printer növünün yaradılacağına qərar verir.

Sabah başqa qəbz hazırlayıcısı istəsən, yaradılma seçimi OrderService-in daxilində olduğu üçün həmin sinfə toxunmalısan. **Coupling** bu cür bağlılıqdır: bir hissənin dəyişməsi digər hissəyə də dəyişiklik yayır.

# DI ilə proqram: printer-i kənarda yaradıb veririk

İndi eyni nəticəni alaq, amma Printer-in yaradılmasını OrderService-dən çıxaraq. Bu da tam alternativ `Program.cs` nümunəsidir.

C# - Program.cs, constructor injection

```
var printer = new Printer();
var orders = new OrderService(printer);
Console.WriteLine(orders.Complete());

public class Printer
{
    public string Print()
    {
        return "Qebz hazirdir";
    }
}

public class OrderService
{
    private Printer _printer;

    public OrderService(Printer printer)
    {
        _printer = printer;
    }

    public string Complete()
    {
        return _printer.Print();
    }
}
```

Nəticə yenə **Qebz hazirdir** olur. Printer-i indi proqramın başlanğıcındakı kod yaradır və OrderService-ə verir. **Bu artıq DI-dır; ayrıca container istifadə etməmişik.**

# Constructor injection obyekt yaratmaqdan necə ayrılır

**Konstruktor - constructor** obyekt yaradılarkən onun ilkin vəziyyətini quran xüsusi hissədir. Siniflə eyni adda yazılır. Əvvəlki nümunədə `OrderService(Printer printer)` konstruktor idi.

C# - əvvəlki proqramın iki sətri

```
var printer = new Printer();
var orders = new OrderService(printer);
```

Birinci `new` Printer obyektini yaradır. İkinci `new` OrderService obyektini yaradır və onun konstruktoruna mövcud Printer-i verir. **Injection ötürmə addımıdır.** Konstruktorun parametrində Printer yazılması öz-özünə Printer yaratmır.

`printer` konstruktora gələn **parametr**, `_printer` isə onu saxlayan sahədir. Parametr çağırış zamanı verilən dəyəri qəbul edir. `_printer = printer` həmin obyektə istinadı sahədə saxlayır.

**İstinad - reference** obyektə çatmağa imkan verən dəyərdir. Sinif obyektini belə ötürəndə obyektin surəti yaradılmır:

Eyni obyektə iki yoldan çatırıq

```
Program-daki printer  ----+
                          +----> Printer obyekti
orders daxilinde _printer-+
```

Məsələn, `var second = printer;` yazsaq da yeni Printer yaranmır. `second` və `printer` eyni obyektə istinad edir. **ReferenceEquals** iki istinadın eyni obyektə aid olub-olmadığını yoxlayır.

Konstruktora `null`, yəni obyekt olmayan boş dəyər də ötürmək mümkündür. Məcburi asılılıq üçün constructor-da bunu yoxlamaq olar. Constructor parametrinin olması təkbaşına bütün yanlış istifadələri aradan qaldırmır.

# Interface hansı qəbz növünün dəyişə biləcəyini göstərir

Əllə DI etdik, amma OrderService hələ konkret Printer sinfini tanıyır. **Interface** tələb olunan əməliyyatları müəyyən edən müqavilədir. Sifarişə konkret cihaz və ya fayl təfərrüatı deyil, qəbz hazırlamaq əməliyyatı lazımdır.

C# - ayrıca tip tərifləri

```
public interface IPrinter
{
    string Print();
}

public class Printer : IPrinter
{
    public string Print()
    {
        return "Adi qebz";
    }
}

public class PdfPrinter : IPrinter
{
    public string Print()
    {
        return "PDF qebzi";
    }
}
```

`IPrinter` müqaviləsi mətn qaytaran Print əməliyyatı tələb edir. `: IPrinter` həmin müqavilənin yerinə yetirildiyini bildirir. Konkret həllə **implementation - reallaşdırma** deyilir. Printer və PdfPrinter iki reallaşdırmadır.

Interface-in özünü `new IPrinter()` ilə yaratmırıq. Müqaviləni yerinə yetirən konkret sinfin obyektini yaradırıq. Burada PdfPrinter də sadəcə izah üçün mətn qaytarır; real PDF hazırlamır.

# Eyni sifariş koduna iki fərqli printer verək

Əvvəlki səhifənin IPrinter, Printer və PdfPrinter tərifləri layihədə qalır. OrderService isə artıq müqaviləni qəbul edir:

C# - OrderService.cs

```
public class OrderService
{
    private IPrinter _printer;

    public OrderService(IPrinter printer)
    {
        _printer = printer;
    }

    public string Complete()
    {
        return _printer.Print();
    }
}
```

C# - Program.cs

```
var ordinary = new OrderService(new Printer());
var pdf = new OrderService(new PdfPrinter());

Console.WriteLine(ordinary.Complete());
Console.WriteLine(pdf.Complete());
```

Nəticələr ardıcıl **Adi qebz** və **PDF qebzi** olur. OrderService-in daxilini dəyişmədik; ona verilən konkret həlli dəyişdik.

**Loose coupling - zəif bağlılıq** bir hissəni dəyişəndə dəyişikliklərin digər hissələrə daha az yayılmasıdır. Interface konkret printer-in seçimini sifarişin əsas məntiqindən ayırmağa kömək etdi. DI interface olmadan da var idi; interface əlavə dəyişiklik sərbəstliyi verdi.

# Nəzəri fərqi testdə görək

**Test** kodun gözlənilən davranışını yoxlamaqdır. **Fake** isə yoxlama üçün yazılmış sadə əvəzedicidir. Həqiqi printer əvəzinə əvvəlki müqaviləyə uyğun əvəzedici verə bilərik:

C# - FakePrinter.cs

```
public class FakePrinter : IPrinter
{
    public string Print()
    {
        return "Test qebzi";
    }
}
```

C# - Program.cs

```
var orders = new OrderService(new FakePrinter());
Console.WriteLine(orders.Complete());
```

Nəticə **Test qebzi** olur. Printer cihazına və əlavə test kitabxanasına ehtiyac olmadı. Əvəzedicini özümüz verə bildik.

| Məsələ | DI-sız ilk nümunə | Interface və DI ilə |
|---|---|---|
| Obyekti kim yaradır | OrderService | Kənardakı kod |
| Konkret növü kim seçir | OrderService | Kənardakı kod |
| OrderService nəyi tanıyır | Printer sinfini | IPrinter müqaviləsini |
| Test əvəzedicisi | Daxili yaradılmaya müdaxilə tələb edir | Konstruktorla verilə bilir |

**new yoxa çıxmadı.** Obyekt yaratmaq yenə lazımdır. Konkret asılılığı yaratmaq qərarını onu istifadə edən biznes sinfindən kənara çıxardıq. Biznes burada sifarişi tamamlamaq kimi tətbiqin əsas iş qaydasını bildirir.

# Container əllə gördüyümüz birləşdirməni təşkil edir

İki obyekti əllə birləşdirmək asandır. Amma böyük proqramda hər sinfin öz asılılıqları ola bilər. Bu əlaqələri bir yerə toplamaq üçün **DI container** istifadə edilir.

Container hansı xidmətin necə təmin olunacağını, hansı hallarda mövcud obyektin paylaşılacağını və idarə etdiyi resursların nə vaxt təmizlənəcəyini müəyyən edən qeydiyyatlarla işləyir.

**Xidmət - service** burada başqa kodun istifadə etdiyi köməkçi obyektdir. Printer və OrderService xidmət ola bilər. Bu söz ayrıca server və ya şəbəkədə işləyən mikroservis mənasına gəlmir.

OrderService tələb olunanda

```
1. Onun konstruktorundaki IPrinter ehtiyaci gorulur.
2. Qeydiyyatdan IPrinter-in konkret helli tapilir.
3. Helle uygun Printer obyekti elde edilir.
4. Printer konstruktorla OrderService-e verilir.
5. Hazir OrderService istifade edecek koda qaytarilir.
```

Asılılığın özünün də constructor ehtiyacları varsa onlar da bu qayda ilə təmin olunur. Bu əlaqələrə **object graph - obyektlər əlaqəsi** deyilir. Bu nümunədə əlaqə sadədir: OrderService Printer-dən istifadə edir.

**DI ötürmə üsuludur; container bu ötürmələri təşkil edən vasitədir.** Container olmadan yazdığımız constructor injection da tam DI idi. Container-dən istifadədə isə sinif ehtiyacını bildirir, yaratma və paylaşma qaydaları kənarda saxlanılır.

# Scope-dan əvvəl HTTP sorğusunu anlayaq

**Client - müştəri proqramı** işi istəyən tərəfdir: brauzer və ya telefondakı tətbiq. **Server** bu istəyi qəbul edib cavab verən tərəfdir.

**Request - sorğu** client-in serverə göndərdiyi iş istəyidir. **Response - cavab** serverin qaytardığı nəticədir. HTTP bu sorğu və cavabların göndərilmə qaydalarını müəyyən edir.

Bir HTTP sorğusu

```
Telefon: "Sifarislerimi goster."
    |
    v
Server: istifadecini mueyyen edir.
        bazadan sifarisleri oxuyur.
        cavabi hazirlayir.
    |
    v
Telefon: neticeni ekranda gosterir.
```

**ASP.NET Core** C# ilə veb server tətbiqi qurmaq üçün istifadə olunan çərçivədir. O, sorğunun uyğun koda çatdırılması və cavabın göndərilməsi kimi işləri təşkil edir.

Eyni istifadəçi əvvəl menyunu, sonra sifarişlərini istəyə bilər. Bunlar iki ayrı HTTP sorğusudur. Sorğu istifadəçinin bütün sistemdə qalma müddəti deyil. Eyni vaxtda müxtəlif istifadəçilərin sorğuları da işlənə bilər.

İndi bir sorğunun başlanğıcından sonuna qədər istifadə olunan xidmətlərin idarə edilməsinə sərhəd qoymaq ehtiyacı yaranır. Növbəti səhifədə bu sərhədə scope deyəcəyik.

# Scope bir iş üçün açılan idarəetmə sərhədidir

**Scope** müəyyən iş ərzində xidmətlərin paylaşılmasına və təmizlənməsinə sərhəd qoyan sahədir. Adi ASP.NET Core HTTP tətbiqində framework hər yeni sorğu üçün ayrıca scope açır.

Birinci sorğu

```
Scope A acilir.
    Istifadeci mueyyen edilir.
    Sifarisler oxunur.
    Cavab hazirlanir.
Scope A baglanir.

Ikinci sorğu
Scope B acilir.
    Yeni sorgunun isi gorulur.
Scope B baglanir.
```

**Hər HTTP sorğusuna ayrıca scope** deyəndə adi sorğu emalı nəzərdə tutulur. Proqramçı ayrıca iş sərhədi üçün əlavə scope da aça bilər. Beləliklə, scope anlayışı yalnız HTTP-yə aid deyil.

Scope xidmətləri idarə etmək üçün sərhəddir. Hansı xidmətin scope daxilində eyni obyekt olacağını isə ayrıca paylaşılma qaydası müəyyən edir. Ona görə **yeni scope açılması bütün xidmət obyektlərinin mütləq yenidən yaradılması demək deyil**.

Scope bağlananda onun idarə etdiyi təmizlənmə işləri görülür. Bu, bütün obyektlərin həmin saniyə yaddaşdan silinməsi mənasına gəlmir. Resursun buraxılması ilə yaddaşın geri alınmasını dərsin sonrakı hissəsində ayıracağıq.

# HTTP scope-unu EF Core deyil ASP.NET Core açır

**EF Core** C# obyektləri ilə verilənlər bazası arasında işləməyi asanlaşdıran kitabxanadır. **DbContext** baza sorğuları və obyekt dəyişiklikləri ilə işləmək üçün onun əsas obyektidir.

Adi veb tətbiqində HTTP sorğusunun scope-unu ASP.NET Core açır. `AddDbContext` ilə qeyd edilən DbContext standart olaraq həmin scope daxilində paylaşılır. Bu qeydiyyat qaydasının adı scoped-dir; onu az sonra ayrıca nümunə ilə yoxlayacağıq.

Bir HTTP sorğusu və onun scope-u

```
Scope A acilir.
    DbContext elde edilir.
    SQL sorqusu 1: sifarisleri oxu.
    SQL sorqusu 2: mehsullari oxu.
    Cavab hazirlanir.
Scope A baglanir.
```

**SQL** baza ilə işləmək üçün sorğu dilidir. Bir HTTP sorğusunda bir neçə SQL sorğusu ola bilər. Hər SQL sorğusu üçün ayrıca HTTP scope-u açılmır.

Console proqramında, yəni terminalda işləyən tətbiqdə, HTTP sorğusunun avtomatik sərhədi yoxdur. Orada işi təşkil edən kod ayrıca scope aça bilər. Fon işində də sərhədi müəyyən etmək lazımdır. **Fon işi** istifadəçinin hazırkı HTTP sorğusundan ayrı emal olunan işdir.

Buradakı nəticə: **scope bir işin idarəetmə sərhədidir; onu kimin açması tətbiqin necə işləməsindən asılıdır.**

# Resolve container-dən xidmət əldə etməkdir

İndi adı qoya bilərik: **resolve** container-dən qeydiyyata uyğun xidmət istəmək və onu əldə etməkdir. Container lazım gələrsə obyekti və onun asılılıqlarını yaradır, qayda icazə verirsə mövcud obyekti qaytarır.

C# - artıq açılmış scope üçün fraqment

```
var printer =
    scope.ServiceProvider.GetRequiredService<Printer>();
```

`scope` bizim açdığımız iş sərhədidir. `scope.ServiceProvider` həmin scope daxilində xidmət təmin edən mexanizmdir. **IServiceProvider** bu təminetmə imkanını ifadə edən .NET müqaviləsidir.

`GetRequiredService<Printer>()` Printer istədiyimizi bildirir. `<Printer>` burada tələb olunan tipi göstərir. Printer qeydiyyatda yoxdursa GetRequiredService xəta verir. `GetService<Printer>()` isə qeydiyyat tapılmayanda boş nəticə verə bilər.

Üç əməliyyatı qarışdırma:

- `new Printer()` obyekti birbaşa yaradır.
- `GetRequiredService<Printer>()` onu container-dən istəyir. Yeni və ya mövcud obyektin verilməsini qeydiyyat müəyyən edir.
- `printer.Print()` artıq alınmış obyektin metodunu çağırır. Bu ayrıca resolve deyil.

OrderService constructor-da Printer istəyəndə də container həmin asılılığı təmin edir. Buna görə adi biznes sinfinin hər metodunda container axtarmağa ehtiyac yoxdur. Nümunələrdə açıq GetRequiredService çağırışı prosesi görmək üçündür.

# Qeydiyyat siyahısı provider və lifetime ayrı şeylərdir

**ServiceCollection** xidmətlərin necə təmin ediləcəyinə dair qeydiyyat siyahısıdır. **BuildServiceProvider** bu siyahıdan xidmət təmin edən container qurur. **CreateScope** onun daxilində ayrıca iş sərhədi açır.

**Lifetime** müəyyən qeydiyyat üçün yeni obyekt yaratma və mövcud obyekti paylaşma qaydasıdır. Üç əsas alternativ var:

C# - bunlardan yalnız birini seçirik

```
services.AddTransient<Printer>();
services.AddScoped<Printer>();
services.AddSingleton<Printer>();
```

Transient hər ayrıca resolve üçün yeni nüsxə, scoped eyni scope üçün bir nüsxə, singleton həmin container və qeydiyyat üçün bir nüsxə paylaşır. Sonrakı səhifələrdə bunları ayrıca izləyəcəyik.

C# - hazırlıq fraqmenti

```
var services = new ServiceCollection();
services.AddScoped<Printer>();

using var provider = services.BuildServiceProvider();
using var scope = provider.CreateScope();

var printer =
    scope.ServiceProvider.GetRequiredService<Printer>();
```

`AddScoped` burada Printer yaratmır; qaydanı qeyd edir. Provider qurulması da bu nümunədə Printer yaratmır. Printer ilk dəfə tələb olunanda yaradılır.

`using var` dəyişənin aid olduğu kod sahəsindən çıxanda onun Dispose təmizləməsini çağırmağı təşkil edir. Bu nümunədə scope provider-dən sonra yaradıldığı üçün çıxışda əvvəl scope, sonra provider bağlanır. Dispose-un mənasını ayrıca təmizlənmə hissəsində geniş açacağıq.

# Transient eyni scope-da da ayrı obyektlər verir

C# - qeydiyyat

```
services.AddTransient<Printer>();
```

Tutaq ki, iki ayrı scope açılıb. Hərəsindən iki dəfə Printer istəyirik:

C# - dörd ayrıca resolve

```
var a = scope1.ServiceProvider.GetRequiredService<Printer>();
var b = scope1.ServiceProvider.GetRequiredService<Printer>();
var c = scope2.ServiceProvider.GetRequiredService<Printer>();
var d = scope2.ServiceProvider.GetRequiredService<Printer>();
```

Hər nömrə ayrı obyektdir

```
Scope 1: a -> obyekt 1
         b -> obyekt 2

Scope 2: c -> obyekt 3
         d -> obyekt 4

Cemi: 4 Printer obyekti.
```

Transient qaydası eyni scope daxilində də əvvəlki nüsxəni paylaşmır. Dörd ayrıca resolve dörd nüsxə verdi.

Amma `a.Print(); a.Print();` yazsaq, artıq aldığımız eyni a obyektində iki metod çağırışı edərik. **Transient hər metod çağırışında yenilənmək deyil.**

Transient ayrıca alınan xidmətin müstəqil nüsxəsi lazım olanda uyğun ola bilər. Seçimi yalnız “yüngül sinifdir” fikri ilə etmə; obyektin vəziyyətini kimlərin paylaşmalı olduğunu da nəzərə al.

# Scoped eyni scope daxilində eyni obyekti verir

C# - qeydiyyat

```
services.AddScoped<Printer>();
```

Əvvəlki səhifədəki eyni dörd resolve çağırışı indi başqa nəticə verir:

İcra ardıcıllığı

```
Scope 1:
    a istenir -> obyekt 1 yaradilir.
    b istenir -> obyekt 1 qaytarilir.

Scope 2:
    c istenir -> obyekt 2 yaradilir.
    d istenir -> obyekt 2 qaytarilir.

Cemi: 2 Printer obyekti.
```

Birinci scope-da a ilə b eyni obyektə istinad edir. İkinci scope-da c ilə d eyni obyektə istinad edir. a ilə c isə ayrıdır.

Adi HTTP tətbiqində sifariş və ödəniş xidmətləri eyni sorğunun scope-undan eyni scoped Printer qeydiyyatını istəsə, eyni printer-i ala bilər. Növbəti sorğunun scope-u ayrıca printer alar.

**Scoped “bir istifadəçiyə bir obyekt” demək deyil.** Eyni istifadəçinin iki ayrı sorğusu adətən iki ayrı scope-dur. Scoped “bir thread-ə bir obyekt” də demək deyil. Thread kod icra edən axındır; scope-un sərhədi həmin axının adı ilə müəyyən edilmir.

DbContext-in `AddDbContext` ilə standart scoped qeydiyyatı bir iş daxilində onun paylaşılmasına kömək edir. Eyni DbContext-i eyni anda paralel baza əməliyyatlarında istifadə etmək isə uyğun deyil; scoped olmaq onu avtomatik təhlükəsiz etmir.

# Singleton müxtəlif scope-larda da eyni obyekti verir

C# - qeydiyyat

```
services.AddSingleton<Printer>();
```

Eyni dörd resolve çağırışı bu dəfə yalnız bir Printer obyekti ilə işləyir:

İcra ardıcıllığı

```
Scope 1:
    a istenir -> obyekt 1 yaradilir.
    b istenir -> obyekt 1 qaytarilir.

Scope 2:
    c istenir -> obyekt 1 qaytarilir.
    d istenir -> obyekt 1 qaytarilir.

Cemi: 1 Printer obyekti.
```

Burada singleton ilk dəfə istənəndə yaranır. Sonrakı scope-lar da həmin container və qeydiyyat üçün eyni nüsxəni alır. Başqa container və ya başqa server prosesində ayrıca singleton ola bilər. **Proses** əməliyyat sistemində işləyən proqram nüsxəsidir.

Singleton sorğular arasında paylaşılır. Buna görə içində “hazırkı istifadəçinin sifarişi” kimi dəyişən məlumat saxlamaq istifadəçilərin məlumatını qarışdıra bilər.

**Thread-safe** eyni anda istifadə zamanı düzgün davranışı qorumaq deməkdir. Container-in özü xidmət təmin etməyi təhlükəsiz təşkil edə bilər, amma yaratdığı sinfin daxilindəki dəyişiklikləri avtomatik təhlükəsiz etmir. Singleton seçimi paylaşılma ehtiyacına və paralel istifadəyə görə əsaslandırılmalıdır.

Bir nüsxə yaratmaq həmişə daha düzgün dizayn demək deyil. Hansı məlumatın kimlərlə paylaşılacağını əvvəl müəyyən et.

# Eyni obyekt olmasını ReferenceEquals ilə yoxlayaq

`ReferenceEquals(a, b)` a və b-nin eyni obyektə istinad edib-etmədiyini yoxlayır. **True** doğru, **False** yanlış nəticədir. Printer daxilindəki mətnlərin bərabər olması obyektlərin eyni olması demək deyil.

C# - dörd resolve-dan sonra yoxlama

```
Console.WriteLine(ReferenceEquals(a, b));
Console.WriteLine(ReferenceEquals(c, d));
Console.WriteLine(ReferenceEquals(a, c));
```

| Lifetime | a ilə b | c ilə d | a ilə c |
|---|---|---|---|
| Transient | False | False | False |
| Scoped | True | True | False |
| Singleton | True | True | True |

Birinci iki sütun eyni scope daxilini yoxlayır. Son sütun isə ayrı scope-ların aldığı obyektləri müqayisə edir.

Hər hərf ayrı obyekt deməkdir

```
Transient: Scope 1 -> A, B    Scope 2 -> C, D
Scoped:    Scope 1 -> A, A    Scope 2 -> B, B
Singleton: Scope 1 -> A, A    Scope 2 -> A, A
```

Bu cədvəl eyni container-də eyni qeydiyyatın dörd dəfə ayrıca istənməsi üçündür. Müxtəlif qeydiyyatların eyni obyektə bağlanması ayrıca qərardır. Ona görə “eyni sinif həmişə eyni singleton-dur” kimi ümumi hökm vermirik.

# Console təcrübəsini işə salmaq üçün hazırlıq

**.NET SDK** C# layihəsini qurmaq və işlətmək üçün alətlər toplusudur. `dotnet` həmin alətlərin terminaldan çağırıldığı əmrdir. Bu dərsin nümunələri .NET 10 ilə yoxlanır.

Yeni Console layihəsi yaratmaq və DI kitabxanasını əlavə etmək üçün:

Terminal əmrləri

```
dotnet new console -n DiDemo
cd DiDemo
dotnet add package Microsoft.Extensions.DependencyInjection
```

**Paket - package** layihəyə əlavə edilən hazır kitabxanadır. Buradakı paket ServiceCollection və digər DI imkanlarını təqdim edir. `using Microsoft.Extensions.DependencyInjection;` isə həmin adları kodda qısa istifadə etməyə imkan verir.

Növbəti səhifədəki tam kodu layihənin **Program.cs** faylına yaz. Sonra işlət:

Terminal əmri

```
dotnet run
```

Əvvəl AddTransient ilə işlədirik. Sonra yalnız həmin qeydiyyat sətrini AddScoped ilə əvəz edib yenidən işlədirik. Üçüncü dəfə AddSingleton ilə yoxlayırıq. Üç qeydiyyatı birlikdə əlavə etmirik.

Kodda scope-ları əllə açmağımızın səbəbi Console proqramında avtomatik HTTP sorğusu olmamasıdır. Bu iki sərhəd lifetime fərqini ayrıca müşahidə etməyə imkan verir.

# Console üçün tam işləyən lifetime proqramı

Bu variantda Printer-in boş sinif olması kifayətdir. Məqsəd qəbzi deyil, obyektlərin eyni və ya ayrı olmasını yoxlamaqdır.

C# - Program.cs

```
using System;
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection();
services.AddTransient<Printer>();

using var provider = services.BuildServiceProvider();
using var scope1 = provider.CreateScope();
using var scope2 = provider.CreateScope();

var a = scope1.ServiceProvider.GetRequiredService<Printer>();
var b = scope1.ServiceProvider.GetRequiredService<Printer>();
var c = scope2.ServiceProvider.GetRequiredService<Printer>();
var d = scope2.ServiceProvider.GetRequiredService<Printer>();

Console.WriteLine("AB: " + ReferenceEquals(a, b));
Console.WriteLine("CD: " + ReferenceEquals(c, d));
Console.WriteLine("AC: " + ReferenceEquals(a, c));

public class Printer
{
}
```

`AB` birinci scope daxilində a ilə b-ni, `CD` ikinci scope daxilində c ilə d-ni, `AC` isə ayrı scope-lardan a ilə c-ni müqayisə edir. + burada mətnləri birləşdirir.

`using var` sətirləri iki scope-un və əsas provider-in proqramın işi bitəndə bağlanmasını təşkil edir. Eyni anda açıq iki scope olması onların bir-birinə qarışması demək deyil.

# Proqramın üç ayrı işə salınmasının nəticəsi

Tam proqramda qeydiyyat sətrini dəyişərək alınan nəticələr:

AddTransient<Printer>() ilə

```
AB: False
CD: False
AC: False
```

Dörd resolve, dörd obyekt. Scope-lar eyni olsa da hər ayrıca resolve yeni nüsxə alır.

AddScoped<Printer>() ilə

```
AB: True
CD: True
AC: False
```

Dörd resolve, iki obyekt. Hər scope bir nüsxəni daxilində paylaşır.

AddSingleton<Printer>() ilə

```
AB: True
CD: True
AC: True
```

Dörd resolve, bir obyekt. Scope-lar arasında da həmin nüsxə paylaşılır.

Buradakı nəticələr dörd ayrıca GetRequiredService çağırışı üçün nəzərdə tutulub. Əgər `var b = a;` yazsan, b sadəcə a-dakı istinadı alar. Hətta transient qeydiyyatda da bu zaman ReferenceEquals(a, b) True olar, çünki ikinci resolve etməmisən.

# İki dəyişən və iki metod çağırışı yeni resolve deyil

İstinadı ötürməklə yeni obyekt yaratmağın fərqini ayrıca yoxlayaq. Burada adi qəbz hazırlayan Printer tərifindən istifadə olunur:

C# - artıq açılmış scope daxilində

```
var a = scope.ServiceProvider.GetRequiredService<Printer>();
var b = a;

Console.WriteLine(ReferenceEquals(a, b));
Console.WriteLine(a.Print());
Console.WriteLine(a.Print());
```

Birinci nəticə **True** olur. Sonrakı iki sətir eyni a obyektinin Print metodunu çağırır. Container-dən yalnız bir dəfə xidmət istəmişik.

Bu iki sətir isə iki resolve-dur

```
var a = scope.ServiceProvider.GetRequiredService<Printer>();
var b = scope.ServiceProvider.GetRequiredService<Printer>();
```

Bu halda eyni və ya ayrı obyektin verilməsini lifetime müəyyən edir.

Eyni fərq constructor injection üçün də keçərlidir. Bir OrderService obyektinə printer verilib sahədə saxlanıbsa, onun Complete metodunu təkrar çağırmaq həmin sahəni avtomatik başqa printer-lə əvəz etmir.

**Obyektin mövcud olması ilə obyekt üzərində əməliyyat çağırılması ayrı hadisələrdir.** Yeni resolve görmədən “burada transient olduğu üçün dəyişəcək” nəticəsinə gəlmə.

# Interface və OrderService-i container ilə birləşdirək

Əvvəldə IPrinter, Printer, PdfPrinter və OrderService-i əllə birləşdirdik. İndi həmin siniflər ayrı fayllarda layihədə qalsın. Aşağıdakı Program.cs onların container vasitəsilə qurulmasını göstərir:

C# - Program.cs, əvvəlki interface və siniflərlə

```
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection();
services.AddScoped<IPrinter, Printer>();
services.AddScoped<OrderService>();

using var provider = services.BuildServiceProvider();
using var scope = provider.CreateScope();

var orders = scope.ServiceProvider
    .GetRequiredService<OrderService>();

Console.WriteLine(orders.Complete());
```

Nəticə **Adi qebz** olur. Birinci qeydiyyatda soldakı IPrinter tələb olunan müqavilə, sağdakı Printer isə konkret reallaşdırmadır.

OrderService istənəndə container onun IPrinter asılılığını görür. Bu scope üçün Printer yaradır və onu OrderService-in konstruktoruna verir. Eyni scope-da yenidən OrderService istənəndə scoped qaydasına görə həmin OrderService qaytarılır.

`AddScoped<IPrinter, PdfPrinter>()` seçsən, eyni sifariş kodunun nəticəsi **PDF qebzi** olar. Alternativ qeydiyyat əvvəlkini əvəz edir; bu təcrübədə iki sətri birlikdə yazmırıq.

Burada GetRequiredService yalnız proqramın birləşdirmə yerindədir. OrderService-in daxilində container istifadə edilmədi; sinif ehtiyacını yenə constructor-da bildirir.

# Real HTTP sorğusunda scope-u framework açır

Yeni veb layihəsi üçün `dotnet new web -n DiWebDemo` yaz, həmin qovluğa keç və Program.cs-ni aşağıdakı kodla əvəz et. ASP.NET Core-un veb layihəsində DI imkanları hazır təqdim olunur.

C# - veb layihəsinin tam Program.cs faylı

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<Printer>();

var app = builder.Build();

app.MapGet("/printers", (Printer a, Printer b) =>
{
    return new
    {
        A = a.Id,
        B = b.Id,
        Same = ReferenceEquals(a, b)
    };
});

app.Run();

public class Printer
{
    public Guid Id { get; } = Guid.NewGuid();
}
```

`MapGet` həmin ünvana gələn GET sorğusunu qarşılayacaq kodu müəyyən edir. GET məlumat almaq üsuludur. `=>` burada çağırılacaq kod blokunu göstərir. ASP.NET Core qeydiyyatda olan Printer parametrlərini sorğunun scope-undan təmin edir.

`Id` obyekti izləmək üçün verdiyimiz tanıdıcıdır. `Guid.NewGuid()` yeni tanıdıcı dəyər yaradır. `get` onun oxuna bildiyini göstərir. `new { ... }` A, B və Same məlumatları olan sadə nəticə obyekti qurur.

# İki HTTP sorğusunun nəticəsini oxuyaq

Veb layihəsinin qovluğunda aşağıdakı əmri işlət:

Terminal əmri

```
dotnet run --urls http://localhost:5087
```

Brauzerdə `http://localhost:5087/printers` ünvanını aç. `localhost` öz kompüterini, `5087` isə proqramın dinlədiyi portu bildirir. Port şəbəkə əlaqəsinin həmin kompüterdə hansı girişə çatacağını seçir.

Cavabda A və B tanıdıcıları, bir də Same nəticəsi görünəcək. Bu, **JSON**, yəni məlumatı ad və dəyər cütləri ilə yazan mətn formatıdır. **Same** eyni obyekt olub-olmadığını bildirir.

Scoped variantda birinci cavabın A və B dəyərləri eyni olur. Səhifəni yeniləyib ikinci HTTP sorğusunu göndərəndə yenə öz aralarında eyni, amma əvvəlki sorğudakından fərqli dəyərlər alınır.

| Qeydiyyat | Bir cavabda A və B | İkinci sorğunun obyekti |
|---|---|---|
| Transient | Ayrı, Same false | Yeni obyektlər |
| Scoped | Eyni, Same true | Əvvəlki sorğudan ayrı |
| Singleton | Eyni, Same true | Əvvəlki sorğu ilə eyni |

Bu üç variant ayrıca işə salınır. Tətbiqi bağlayıb yenidən açmaq yeni container yaradır; singleton da əvvəlki prosesdəki obyektin davamı olmur.

Kodda CreateScope yazmadıq. **HTTP scope-unu ASP.NET Core özü idarə etdi.** Printer parametrlərinin avtomatik verilməsi də endpoint inteqrasiyasıdır. Endpoint müəyyən ünvan və sorğu üsuluna cavab verən giriş nöqtəsidir.

# Constructor method və property injection fərqi

Constructor injection-da asılılıq obyekt yaranarkən verilir. İlk nümunələrimizdə bunu gördük. Obyektin daimi ehtiyacını constructor-da göstərmək onun nəyə bağlı olduğunu açıq edir.

**Method injection** asılılığın konkret əməliyyata verilməsidir. Aşağıdakı sinif əvvəlki IPrinter müqaviləsindən istifadə edir:

C# - method injection

```
public class ReceiptJob
{
    public string Run(IPrinter printer)
    {
        return printer.Print();
    }
}
```

Çağıran kod `new ReceiptJob().Run(new Printer())` yaza bilər. Printer yalnız həmin çağırışa verilir.

**Property injection** asılılığın obyektin xüsusiyyətinə sonradan verilməsidir. Property oxuma və ya yazma imkanı verən xüsusiyyətdir:

C# - property injection

```
public class ReceiptJob
{
    public IPrinter Printer { get; set; } = null!;
}
```

Çağıran kod obyekt yaradandan sonra `job.Printer = new Printer();` yazır. `set` dəyər yazmağa imkan verir. `null!` yalnız boş dəyərlə bağlı kompilyator xəbərdarlığını susdurur; obyekt yaratmır və təhlükəsizlik təmin etmir.

Standart .NET container adi sinfin property-lərini avtomatik doldurmur, onun hər metoduna da özbaşına parametr vermir. Veb endpoint-ləri kimi xüsusi yerlərdəki dəstək framework-ün ayrıca inteqrasiyasıdır.

# Singleton scoped asılılığı saxlayanda problem yaranır

Tutaq ki, printer-i scoped, hesabat xidmətini singleton qeyd etdik. Hesabat xidməti printer-i constructor-la alıb öz sahəsində saxlayır:

C# - lifetime uyğunsuzluğu olan qeydiyyat

```
services.AddScoped<Printer>();
services.AddSingleton<ReportRunner>();
```

C# - ReportRunner.cs

```
public class ReportRunner
{
    private Printer _printer;

    public ReportRunner(Printer printer)
    {
        _printer = printer;
    }
}
```

Bir iş sərhədi üçün nəzərdə tutulan Printer indi uzun yaşayan ReportRunner tərəfindən tutulur. Buna **captive dependency**, yəni qısa sərhədli asılılığın uzun yaşayan obyekt tərəfindən saxlanması deyilir.

Scope yoxlaması aktivdirsə container belə asılılığı rədd edə bilər. Console təcrübəsində `BuildServiceProvider(new ServiceProviderOptions { ValidateScopes = true })` bu yoxlamanı aktivləşdirir. `true` burada seçimin açıq olduğunu bildirir.

Hesabat sorğuya aiddirsə onu da scoped etmək mümkün həldir. Həqiqətən uzun yaşayan fon xidmətidirsə, hər iş üçün ayrıca scope yaradıb printer-i həmin scope daxilində istifadə etmək olar. Növbəti səhifədə bunu göstərəcəyik.

# Uzun yaşayan xidmət hər iş üçün ayrıca scope aça bilər

**IServiceScopeFactory** yeni scope yaratmaq üçün istifadə olunan .NET xidmətidir. Burada ReportRunner bir işə başlayanda scope açır, printer-i alır və iş bitəndə scope-u bağlayır. Printer-in Print metodu əvvəlki kimi mətn qaytarır.

C# - ReportRunner.cs, düzgün alternativ

```
public class ReportRunner
{
    private IServiceScopeFactory _scopeFactory;

    public ReportRunner(IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    public string Run()
    {
        using var scope = _scopeFactory.CreateScope();
        var printer = scope.ServiceProvider
            .GetRequiredService<Printer>();

        return printer.Print();
    }
}
```

Bu sinif üçün `using Microsoft.Extensions.DependencyInjection;` lazımdır. Qeydiyyatlar yenə Printer üçün scoped, ReportRunner üçün singleton ola bilər.

İki ayrı Run çağırışı iki ayrı scope açır. Hər çağırış öz scoped printer-ini alır. Printer singleton-un sahəsində saxlanılmır və scope-dan kənarda istifadə üçün qaytarılmır; nəticə olan mətn qaytarılır.

Asinxron təmizlənmə tələb edən resurslarla uyğun `CreateAsyncScope` və `await using` forması istifadə edilə bilər. Asinxron əməliyyat tamamlanmanı gözləyərkən icra axınını bloklamamağa imkan verir. Bunun təfərrüatı ayrıca async dərsindədir.

# Singleton transient obyekti saxlayanda o da dəyişmir

Əvvəlki ReportRunner-in printer-i sahədə saxlayan constructor variantına qayıdaq. Bu dəfə qeydiyyat belə olsun:

C# - bu iki qeydiyyat ayrıca ssenaridir

```
services.AddTransient<Printer>();
services.AddSingleton<ReportRunner>();
```

ReportRunner ilk dəfə yarananda container onun üçün bir Printer yaradır. ReportRunner həmin istinadı sahədə saxlayır. Sonrakı metod çağırışları eyni ReportRunner və eyni printer-lə işləyir.

Nə baş verir

```
ReportRunner ilk defe istenir.
    Printer A yaradilir ve ona verilir.

ReportRunner yeniden istenir.
    Movcud ReportRunner qaytarilir.
    Onun sahesinde hele Printer A var.
```

Printer-in transient qeydiyyatı mövcud sahəni avtomatik yeniləmir. Yeni printer almaq üçün ayrıca resolve və ya uyğun yaradılma mexanizmi lazımdır.

Singleton-un transient asılılıq alması hər halda avtomatik xəta deyil. Amma asılılığın faktiki istifadə müddəti uzanır və eyni vaxtda istifadəyə uyğunluğu da düşünülməlidir. **Qeydiyyat adı ilə obyektin real istifadə müddətini ayrı yoxla.**

Singleton-un hər iş üçün yeni scope açdığı əvvəlki həll printer-in paylaşılma sərhədini açıq idarə edirdi. Seçim obyektin hansı işə aid olmasına görə verilir.

# Dispose resursu buraxır GC yaddaşı geri alır

Fayl və baza bağlantısı kimi resurslar iş bitəndə buraxılmalıdır. **Dispose** bu təmizləmə əməliyyatının adıdır. **IDisposable** onu təqdim edən müqavilədir. `using` çıxışda Dispose çağırılmasını təşkil edir.

Container-in yaratdığı və təmizlənməsini idarə etdiyi, IDisposable olan xidmətlər üçün:

| Xidmət | Təmizlənmə sərhədi |
|---|---|
| Scope-dan alınmış transient | Həmin scope bağlananda |
| Scoped | Həmin scope bağlananda |
| Singleton | Əsas provider bağlananda |

Hər xidmətin Dispose metodu olmur. Sadə boş Printer sinfimizdə belə resurs yoxdur. Təmizlənmə qaydası resurs idarə edən xidmətlərdə əhəmiyyətlidir.

Container-in verdiyi ortaq asılılığı onu istifadə edən hər sinif ayrıca bağlamamalıdır. Öz yaratdığın scope-u isə bağlamalısan. Root, yəni əsas provider-dən alınmış disposable transient-lər provider bağlanana qədər saxlanıla bilər; belə xidmətlər üçün uyğun scope istifadə et.

**GC - Garbage Collector** .NET-in artıq əlçatmaz olan idarə olunan obyektlərin yaddaşını sonradan geri alan mexanizmidir. İdarə olunan dedikdə yaddaşa .NET-in nəzarəti nəzərdə tutulur. Dispose çağırılması obyektin dərhal yaddaşdan silinməsi deyil.

Hazır obyekt özün yaradılıb `AddSingleton(hazirObyekt)` ilə verilirsə, onun təmizlənmə məsuliyyəti də səndə qalır. **IAsyncDisposable** təmizlənməsi asinxron gözlənə bilən resursların müqaviləsidir; burada uyğun await using işlədilir.

# Bir müqavilə üçün bir neçə həll qeyd ediləndə

Əvvəlki IPrinter, Printer və PdfPrinter təriflərini istifadə edək:

C# - iki ayrı reallaşdırmanın qeydiyyatı

```
services.AddTransient<IPrinter, Printer>();
services.AddTransient<IPrinter, PdfPrinter>();
```

Standart .NET container-də tək IPrinter istənəndə bu sadə qeydiyyatların sonuncusu, yəni PdfPrinter seçilir. İkisini birlikdə almaq üçün **IEnumerable<IPrinter>** istənilə bilər. Bu, IPrinter obyektləri ardıcıllığını ifadə edən müqavilədir.

C# - açılmış scope daxilində

```
var printers = scope.ServiceProvider
    .GetRequiredService<IEnumerable<IPrinter>>();

foreach (var printer in printers)
{
    Console.WriteLine(printer.Print());
}
```

`foreach` ardıcıllığın hər elementi üçün daxildəki kodu işləyir. Nəticədə əvvəl **Adi qebz**, sonra **PDF qebzi** yazılır. Eyni xidmət üçün təsadüfən bir neçə qeydiyyat əlavə etmək gözlədiyin seçimi dəyişə bilər.

**Factory - yaradıcı funksiya** obyektin necə yaradılacağını kodla bildirə bilər. Məsələn, ayrıca alternativ qeydiyyat `services.AddTransient<Printer>(provider => new Printer());` yazıla bilər. `provider` funksiya parametridir; burada istifadə edilmir. Container yeni nüsxə tələb olunanda bu funksiyanı çağırır.

Factory qeydiyyatı constructor asılılıqlarını qarışıq şəkildə biznes metodlarının içində axtarmağa ehtiyac yaratmır. Yaradılma qərarı yenə tətbiqin qurulduğu sərhəddə saxlanılır.

# DI IoC və DIP eyni sualı cavablandırmır

Bu üç anlayışı artıq gördüyümüz kodla bağlayaq:

| Anlayış | Bizim nümunədə mənası |
|---|---|
| DI - Dependency Injection | Printer obyektini OrderService-ə kənardan veririk. |
| IoC - Inversion of Control | Yaradılma və əlaqələndirmənin idarəsini container təşkil edir. |
| DIP - Dependency Inversion Principle | Sifarişin əsas məntiqini konkret printer detalına deyil IPrinter müqaviləsinə bağlayırıq. |

**IoC** idarəetmənin müəyyən hissəsinin başqa mexanizmə keçməsi barədə daha geniş anlayışdır. DI onun asılılıqları verməyə aid üsuludur.

**DIP** SOLID prinsiplərinin D hərfidir. SOLID dəyişiklikləri idarə etməyə kömək edən beş dizayn prinsipinin qısa adıdır. DIP-də əsas iş qaydası ilə texniki detal sabit müqaviləyə bağlanır. Burada sifariş qaydası əsas iş, printer-in konkret quruluşu texniki detaldır.

**Abstraksiya** lazımi davranışı detallardan ayıran təsvir və ya müqavilədir. Bizim nümunədə IPrinter bu rolu oynayır. Interface parametri yazıb metodun içində yenidən konkret Printer yaratmaq verilmiş asılılığın faydasını itirə bilər.

DI avtomatik olaraq bütün dizayn problemlərini həll etmir. Hər sadə dəyər və ya kiçik sinif üçün ayrıca interface yaratmaq məcburi deyil. Dəyişən texniki asılılıqları və paylaşılma ehtiyaclarını müəyyən etmək daha vacibdir.

# Tez rast gəlinən problemləri səbəbi ilə tanı

**Qeydiyyat çatışmır.** Constructor IPrinter istəyir, amma container onun necə təmin olunacağını bilmir. Uyğun qeydiyyatı əlavə et. Sinif faylının mövcud olması təkbaşına xidmət qeydiyyatı deyil.

**Dairəvi asılılıq.** A yaratmaq üçün B, B yaratmaq üçün A lazımdır. Buna circular dependency deyilir. Container məsuliyyətləri sənin yerinə bölmür. Hansı kodun hansı işi görməli olduğunu yenidən düşün.

**Gizli asılılıq.** Sinif constructor-da ehtiyaclarını göstərmək əvəzinə hər metodda provider-dən xidmət axtarır. Buna Service Locator yanaşması deyilir. Ehtiyaclar gizlənir və sinfi əllə qurub yoxlamaq çətinləşir. Adi biznes sinfində constructor injection daha açıqdır.

**Sorğudan uzun yaşayan iş.** HTTP sorğusunda alınmış DbContext-i nəzarətsiz fon işinə verirsən, sorğu isə bitir. Scope bağlananda iş artıq təmizlənmiş resursla qarşılaşa bilər. Fon işinin öz scope-u və idarə olunan tamamlanması olmalıdır.

**Bir neçə əsas container.** Qeydiyyat zamanı lazımsız BuildServiceProvider çağırışları ayrıca provider-lər yaradıb fərqli singleton nüsxələri və təmizlənmə sərhədləri yarada bilər. ASP.NET Core tətbiqində əsas container-i adətən builder.Build qurur.

**Çoxlu constructor asılılığı.** Bir sinif on fərqli xidmət tələb edirsə, onun üzərinə çox iş yığılmış ola bilər. Təkcə constructor-u qısaltmaq yox, məsuliyyətləri aydın bölmək barədə düşün.

# Özünü nümunə üzərində yoxla

**1. Constructor-a Printer yazdım. Obyekt avtomatik yarandı?** Xeyr. Əllə istifadədə onu çağıran kod yaradır və verir. Container istifadəsində qeydiyyata uyğun təmin edilir.

**2. DI etmək üçün mütləq interface lazımdır?** Xeyr. Konkret Printer-i kənardan vermək də DI-dır. Interface dəyişə bilən həlli müqavilə arxasında saxlamağa kömək edir.

**3. AddScoped sətrində Printer yaranır?** Bu tip qeydiyyat nümunəsində xeyr. Qayda yazılır; nüsxə xidmət tələb olunanda yaranır.

**4. İki HTTP sorğusu neçə scope-dur?** Adi ASP.NET Core sorğu emalında iki ayrı scope. Eyni istifadəçinin göndərməsi bunu dəyişmir.

**5. Scoped printer-i eyni scope-dan iki dəfə aldım.** Eyni obyekt alınır. Ayrı scope-dan aldıqda ayrıca obyekt alınır.

**6. Transient printer-i bir dəfə alıb Print-i beş dəfə çağırdım.** Bir obyekt üzərində beş metod çağırışı etdin. Beş resolve etmədin.

**7. Singleton hər serverdə eyni obyekt olur?** Xeyr. Hər prosesin və container-in öz qeydiyyat sərhədi var.

**8. Scope-u EF Core açır?** Adi HTTP scope-unu ASP.NET Core açır. DbContext həmin sərhəddə istifadə edilən xidmət ola bilər.

**9. Eyni DbContext eyni scope-da olduğuna görə paralel istifadə təhlükəsizdir?** Xeyr. Scoped olmaq paralel istifadəyə avtomatik uyğunluq vermir.

**10. Mənə “DI-ni izah et” deyilsə:** sinif ehtiyac duyduğu obyekti kənardan alır; container bunu qeydiyyata görə təşkil edə bilər; scope iş sərhədidir; resolve xidməti əldə etməkdir; lifetime həmin xidmətdə nüsxələrin paylaşılma qaydasıdır.

# Mənbələr və nümunələri necə oxumalı

Əsas tədris ardıcıllığı və qəbz nümunələri bu dərs üçün yazılıb. .NET davranışlarını yoxlamaq üçün aşağıdakı rəsmi sənədlərdən istifadə edilir. İlk oxunuşda əlavə mənbələri açmaq məcburi deyil.

- Microsoft Learn - DI və container
- Microsoft Learn - transient scoped singleton
- Microsoft Learn - təmizlənmə və lifetime səhvləri
- Microsoft Learn - ASP.NET Core HTTP sorğusunda DI
- Microsoft Learn - DbContext ömrü və paralel istifadə

**Tam Program.cs** yazılan nümunə həmin layihənin faylını bütövlükdə əvəz edir. **Fraqment** isə artıq hazırlanmış layihənin göstərilən hissəsinə əlavə olunan kod parçasıdır. Ayrıca sinif tərifləri layihədə ayrı fayllarda saxlanıla bilər.

Eyni adlı Printer və OrderService tərifləri alternativ öyrənmə mərhələləridir. Hamısını bir layihəyə üst-üstə əlavə etmək eyni adın təkrar tərifi xətasına səbəb olar. Ekran nəticələri həmin səhifədə göstərilən alternativə aiddir.

Praktik məşq: Console nümunəsində tək qeydiyyat sətrini dəyiş, nəticəni əvvəl özün proqnozlaşdır, sonra işə sal. Daha sonra veb nümunəsində iki ayrı HTTP sorğusunu müqayisə et. Burada məqsəd adları əzbərləmək deyil, hər yeni obyektin hansı addımda yarandığını göstərə bilməkdir.

# 3 Deferred execution, IEnumerable, IQueryable və loading

Bu dərsdə yalnız dörd mövzunu eyni sifariş nümunəsi üzərində öyrənəcəyik:

- **Deferred execution:** sorğunu yazmaqla onu işlədib nəticə almağın fərqi.
- **IQueryable:** EF sorğusuna şərtlərin necə əlavə edilməsi.
- **IEnumerable:** elementləri oxumaq və C#-da filtrləmə; IQueryable ilə fərqi.
- **Loading:** eager, explicit və lazy ilə əlaqəli məhsulların nə vaxt gətirilməsi.

Əvvəl sorğunun özünü, sonra onun nəticəsindəki obyektin əlaqələrini araşdırırıq. Hər kodda iki suala cavab axtar: **SQL hansı sətirdə işləyir? Həmin sətirdən sonra hansı məlumat artıq proqramdadır?**

Nümunələr real SQLite bazasında, EF Core 10.0.9 ilə yoxlanılıb. Hər təcrübə eyni məlumatı olan təzə DbContext ilə başlayır. SQL saylarına başlanğıc məlumatını yaradan əmrlər daxil deyil.

Nümunələrdə `sql.Count` faktiki işlənmiş SQL əmrlərinin sayını göstərir. Kod parçalarını təkrarlamaq üçün işlək layihə mövcuddur. Həmin qovluqda `dotnet run -- query`, `dotnet run -- enumerable`, `dotnet run -- eager`, `dotnet run -- explicit` və `dotnet run -- lazy` əmrləri uyğun nümunəni işlədir.

# Nümunədəki məlumatı tanıyaq

Bazada Orders sifarişləri, OrderItems isə onların məhsul sətirlərini saxlayır. Order 42 Ayselə aiddir; iki məhsulu var: Cay və Tort. Order 43 Murada aiddir, bir məhsulu var. Order 44 Leylaya aiddir, məhsulu yoxdur.

**EF Core** baza məlumatı ilə C# obyektləri arasında işləyən kitabxanadır. **SQL** bazaya sorğu göndərmək üçün dildir. **DbContext** həmin əməliyyatları idarə edən obyektdir; kodda adı `db`-dir. `db.Orders` Order-lar üçün sorğu yazmağa çıxış verir; onların artıq gətirildiyini bildirmir.

C# - dərsdə istifadə olunan obyektin quruluşu

```
public class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; } = "";
    public virtual ICollection<OrderItem> Items { get; set; }
        = new List<OrderItem>();
}
```

**Entity** burada bir baza qeydini təmsil edən C# obyektidir. Order sifarişi, OrderItem isə bir məhsul sətrini təmsil edir. OrderItem-in Id, OrderId, ProductName xassələri var. OrderId onun hansı sifarişə aid olduğunu göstərən nömrədir.

**Navigation property** əlaqəli obyektlərə keçid verən xassədir: Items sifarişin məhsul obyektlərinə keçiddir. `new List<OrderItem>()` sadəcə boş siyahı yaradır; bazadan məhsul oxumur. `virtual` sözünün lazy loading üçün rolunu sonra görəcəyik.

**LINQ** C#-da Where kimi metodlarla sorğu yazmaq imkanıdır. `Where(o => o.Id == 42)` “yalnız Id-si 42 olanları seç” deməkdir. `o` yoxlanılan sifarişi bildirən dəyişəndir; `==` bərabərlik yoxlamasıdır.

# Sorğunu yazmaq hələ onu icra etmək deyil

**Query - sorğu** hansı məlumatı istədiyimizin təsviridir. **Execution - icra** həmin sorğunun işlədilib nəticəsinin alınmasıdır. Aşağıdakı iki sətir fərqli iş görür:

C# - bu iki sətri ayrı-ayrı oxu

```
var query = db.Orders.Where(o => o.Id == 42);

var orders = await query.ToListAsync();
```

Birinci sətir: “Orders-dən yalnız Id-si 42 olanları istəyəcəyəm” qaydasını qurur. Bu mərhələdə **SQL sayı 0-dır**.

İkinci sətir: “İndi həmin qaydanın nəticəsini gətir və siyahı hazırla” deyir. Bu nümunədə **bir SQL** icra olunur. `orders` artıq içində Order obyekti olan List-dir.

SQL - yaranan əmrin sadələşdirilmiş mənası

```
SELECT Id, CustomerName
FROM Orders
WHERE Id = 42;
```

`SELECT` gətiriləcək sütunları, `FROM` cədvəli, `WHERE` şərti bildirir. Provider-in faktiki əmri dırnaqlar və əlavə hissələr baxımından fərqlənə bilər.

**Materialization** gələn baza nəticəsindən C# obyektlərinin hazırlanmasıdır: EF nəticədəki 42 və Aysel dəyərlərini Order obyektinin xassələrinə yerləşdirir.

`ToListAsync` nəticəni siyahıya yığır. Async asinxron əməliyyatı bildirir; `await` nəticəni gözləyib sonra davam etməyə imkan verir. Async ayrıca loading növü deyil.

# Deferred execution nə deməkdir?

**Deferred execution - təxirə salınmış icra** sorğunu qurarkən nəticənin hələ oxunmaması, oxuma işinin nəticəyə ehtiyac yaranan ana saxlanmasıdır.

C# - dotnet run -- foreach

```
var query = db.Orders.Where(o => o.Id == 42);
Console.WriteLine("Evvel SQL: " + sql.Count);

foreach (var order in query)
    Console.WriteLine("Id: " + order.Id);

Console.WriteLine("Sonra SQL: " + sql.Count);
```

Yoxlanmış çıxış

```
Evvel SQL: 0
Id: 42
Sonra SQL: 1
```

`sql` işlək layihədə icra olunmuş SQL əmrlərinin qeydlərini saxlayan siyahıdır. `sql.Count` həmin qeydlərin sayıdır.

`foreach` elementləri bir-bir oxuyan dövrdür. Sorğunun elementlərini oxumaq başlayanda bazaya müraciət edilir. Deməli, **SQL yalnız ToList ilə işləmir; foreach də EF sorğusunu icra edə bilər.**

`Where` və `OrderBy` kimi addımlar EF sorğusunu hazırlayır. `ToListAsync`, `FirstAsync`, `SingleAsync`, `CountAsync`, `AnyAsync` kimi nəticə istəyən əməliyyatlar onu icra edir.

Sorğu obyektinin mövcud olması nəticənin artıq gətirildiyini göstərmir. Sonrakı səhifələrdə məhz bu fərqi qoruyacağıq.

# IQueryable nədir və bizə nə verir?

`IQueryable<Order>` Order sorğusu ilə işləmək üçün **interface**, yəni tipin təmin etdiyi imkanları bildirən müqavilədir. Buradakı `<Order>` sorğunun hansı tip elementlər üzərində qurulduğunu göstərir.

EF sorğusunu bu tipdə saxlayanda sonrakı uyğun Where kimi addımlar da bazaya göndəriləcək sorğunun təsvirinə əlavə oluna bilir:

C# - dotnet run -- query

```
IQueryable<Order> query = db.Orders
    .Where(o => o.Id >= 43);

query = query.Where(o => o.Id == 43);
var rows = await query.ToListAsync();
```

Birinci şərt 43 və 44-ü uyğun sayır. İkinci şərt bunlardan yalnız 43-ü saxlayır. İcra anında hər iki şərt SQL-ə daxil olur. **Bazadan bir Order sətri gəlir: Muradın 43 nömrəli sifarişi.**

**Provider** EF sorğusunu konkret baza üçün SQL-ə çevirən hissədir. EF bu şərtlərin strukturunu saxlayır. Buna **expression tree - ifadə ağacı** deyilir: “Id sütununu 43-lə müqayisə et” kimi əməliyyatların proqram daxilində təsviri. Provider bu təsviri oxuyub uyğun SQL hazırlayır.

IQueryable özü “SQL bazası” demək deyil. Onun davranışını arxasındakı provider müəyyən edir. Bizim nümunədə provider EF Core-un SQLite provider-idir; buna görə uyğun şərtlər SQL-ə çevrilir.

**İki Where yazdıq, amma iki SQL işləmədi.** Sorğunu iki addımda qurub bir dəfə icra etdik.

# IEnumerable nədir? Hazır siyahı deməkdirmi?

`IEnumerable<Order>` “bu mənbənin Order elementlərini bir-bir oxuya bilərsən” müqaviləsidir. Elementləri oxumağa **enumeration** deyilir. Adi foreach bu imkandan istifadə edir.

Bu müqavilə elementlərin haradan gələcəyini demir. Hazır List-dən də gələ bilər, hələ icra edilməmiş EF sorğusundan da. Buna görə **IEnumerable yazılması məlumatın artıq yaddaşda olması demək deyil.**

C# - burada hələ SQL sayı 0-dır

```
IEnumerable<Order> source = db.Orders;
```

Yalnız dəyişənin görünən tipi dəyişdi. Əsl mənbə hələ EF sorğusudur. `foreach` və ya `ToList()` ilə oxunanda bazaya müraciət edilə bilər.

IQueryable<T> həm də IEnumerable<T>-dir. Yəni sorğu qurmaq imkanı olan mənbənin elementlərini oxumaq imkanı da var. Əks istiqamət ümumi qayda deyil: hər IEnumerable SQL qura bilmir.

C# - hazır List nümunəsi

```
List<int> numbers = new() { 1, 2, 3 };
IEnumerable<int> source = numbers;
```

Bu ikinci nümunədə elementlər artıq List-dədir. Heç bir baza yoxdur. IEnumerable hər iki vəziyyətə uyğun gəldiyindən yalnız tipin adına baxıb “SQL işlənib” nəticəsini çıxara bilmərik.

**List<T>** elementləri saxlayan konkret siyahı tipidir. **IEnumerable<T>** isə onları oxumaq imkanını bildirir. Bu fərq sonrakı Where-in harada işləyəcəyinə təsir edir.

# Eyni Where niyə fərqli yerdə işləyir?

İki nümunənin də yekun nəticəsi yalnız 43-dür. Amma bazadan gələn məlumatın miqdarı fərqlənir. Hər variant ayrıca təzə context-də işlədilir.

C# - IQueryable: hər iki şərt SQL-ə gedir

```
IQueryable<Order> query = db.Orders
    .Where(o => o.Id >= 43);
var rows = await query.Where(o => o.Id == 43)
    .ToListAsync();
```

C# - IEnumerable: ikinci şərt C#-da işləyir

```
IEnumerable<Order> source = db.Orders
    .Where(o => o.Id >= 43);
var rows = source.Where(o => o.Id == 43).ToList();
```

Birincidə SQL həm `Id >= 43`, həm `Id == 43` şərtini daşıyır. Baza 43-ü qaytarır. **Bir Order obyekti alınır.**

İkincidə ilk Where tip dəyişməzdən əvvəl yazılıb və SQL-də qalır. Sonrakı Where IEnumerable üçün seçilir: gələn obyektlər C# kodu ilə yoxlanır. **Baza 43 və 44-ü qaytarır; C# 44-ü kənarlaşdırır.**

| Ölçü | IQueryable variantı | IEnumerable variantı |
|---|---|---|
| İcra olunmuş SQL | 1 | 1 |
| Oxunmuş Order obyektləri | 1 | 2 |
| Son siyahıdakı Id | 43 | 43 |

Eyni metod adı müxtəlif müqavilələr üçün təklif olunur. C# hansı variantı seçəcəyinə dəyişənin kodda görünən tipinə görə qərar verir. Burada ikinci variantın Where-i SQL təsviri qurmur; obyekt üzərində işləyən C# yoxlaması olur.

# AsEnumerable və ToList fərqi

`AsEnumerable()` mənbəni IEnumerable kimi davam etdirməyə imkan verir. **Özü SQL işlətmir, özü List yaratmır.** Ondan sonrakı adi LINQ filtrləri gələn obyektlər üzərində C#-da işləyir:

C# - boundary rejimi

```
var query = db.Orders.Where(o => o.Id >= 43)
    .AsEnumerable()
    .Where(o => o.Id == 43);
// SQL: 0

var rows = query.ToList();
// SQL: 1; baza 43 və 44-ü gətirib, C# 43-ü saxlayıb.
```

AsEnumerable-dan əvvəlki Where SQL-də qalır. Sonrakı Where isə SQL-ə əlavə edilmir. ToList mənbəni oxuyub yekun elementləri hazır siyahıda saxlayır.

C# - əvvəl tam siyahını alsaq

```
var list = await db.Orders.ToListAsync();
var filtered = list.Where(o => o.Id == 43).ToList();
```

Bu ikinci nümunədə ilk sətirdə üç sifariş də artıq gəlib. İkinci sətir hazır obyektləri süzür və əlavə Orders SQL-i işləmir.

**Server-side** filtrin bazada hesablanmasıdır. **Client-side** filtrin EF-dən istifadə edən C# proqramında hesablanmasıdır; burada client sözü brauzer demək deyil.

AsEnumerable oxuma qaydasını dəyişir; ToList isə nəticəni həqiqətən oxuyub saxlayır. Bazadan az məlumat gətirmək istəyirsənsə uyğun filtrləri bu keçidlərdən əvvəl yaz.

# Deferred yalnız IQueryable-a aid deyil

Hazır List üzərindəki Where də yoxlamanı nəticə oxunana qədər saxlaya bilər. Bu dəfə heç baza yoxdur:

C# - dotnet run -- list-deferred

```
var numbers = new List<int> { 1, 2 };
IEnumerable<int> query = numbers.Where(n => n > 1);

numbers.Add(3);
Console.WriteLine(string.Join(",", query));
```

Yoxlanmış nəticə

```
2,3
```

Where yazılan anda siyahıda 1 və 2 var idi. Lakin yoxlama hələ yerinə yetirilməmişdi. Sonra 3 əlavə olundu. `string.Join` nəticəni mətnə çevirmək üçün elementləri oxuyanda Where artıq 3-ü də gördü.

Deməli, **IEnumerable həmişə immediate, IQueryable həmişə deferred** kimi əzbərləmək yanlışdır. **Immediate execution** əməliyyat çağırışı zamanı nəticənin hesablanmasıdır; deferred isə nəticənin oxunmasını sonraya saxlayır.

Hazır siyahının özü artıq mövcud məlumatdır. Onun üzərində qurulan yeni Where nəticəsi isə hələ hesablanmamış ola bilər. Mənbənin hazır olması ilə yeni sorğunun icra olunması ayrı hadisələrdir.

Əgər `var result = numbers.Where(n => n > 1).ToList();` yazıb yalnız bundan sonra 3 əlavə etsəydik, result-da yalnız 2 qalardı. Çünki siyahı əvvəlki anda hazırlanmışdı.

# İndi loading sualına keçirik

İndiyədək sorğunun **nə vaxt** icra olunduğunu və filtrin **harada** işlədiyini öyrəndik. İndi başqa sual var: **Order gələndə onun Items məlumatı da gəlirmi?**

Bu dərsdə loading əlaqəli məlumatın gətirilməsidir. Üç üsul həmin məlumatı hansı addımda istədiyimizi fərqli şəkildə ifadə edir:

| Üsul | Məhsulların gətirilməsi necə tələb olunur? |
|---|---|
| Eager loading | Order sorğusuna əvvəlcədən Include əlavə edirik. |
| Explicit loading | Order gələndən sonra ayrıca LoadAsync əmri yazırıq. |
| Lazy loading | Mexanizm aktivdirsə yüklənməmiş Items oxunuşu yükləməni başladır. |

Eager “bu kod sətrini yazan kimi SQL işləyir” demək deyil. Include ilə qurulmuş sorğu da ToListAsync və ya SingleAsync kimi icra addımını gözləyə bilər.

C# - hələ SQL yoxdur

```
var query = db.Orders.Include(o => o.Items);

// İcra burada başlayır.
var order = await query.SingleAsync(o => o.Id == 42);
```

**Deferred execution əsas sorğunun icra anıdır. Lazy loading isə alınmış obyektin əlaqəsini sonradan gətirmək mexanizmidir.** Hər ikisində gecikmə ola bilər, amma gecikdirilən iş eyni deyil.

Sonrakı nümunələrdə lazy yalnız lazy sınağında aktivdir. Avtomatik Include ayarı yoxdur və hər sınaq təzə context ilə başlayır.

# Heç loading istəməsək nə gəlir?

C# - yalnız Order gətiririk

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);

Console.WriteLine(order.CustomerName);
Console.WriteLine(order.Items.Count);
Console.WriteLine(
    db.Entry(order).Collection(o => o.Items).IsLoaded);
```

Yoxlanmış nəticə - lazy bağlıdır

```
Aysel
0
False
SQL: 1
```

Baza Orders-dən 42-ni qaytardı, EF Order obyektini hazırladı. Bu sorğuda OrderItems istənməyib. Sinifdəki `new List<OrderItem>()` boş siyahı yaratdığı üçün Items.Count 0 görünür. **Bazadakı iki məhsul itməyib; sadəcə bu obyektin Items-inə gətirilməyib.**

`Count` burada mövcud kolleksiyadakı element sayıdır. `db.Entry(order)` EF-in həmin obyektlə bağlı idarəetmə məlumatına çıxış verir. `Collection(o => o.Items)` Items əlaqəsini seçir. `IsLoaded` EF-in onu yüklənmiş sayıb-saymadığını bildirir.

False yüklənmiş sayılmadığını göstərir. True və False məntiqi doğru və yanlış dəyərlərdir.

Bu təcrübədə context təzədir və məhsullar əvvəl alınmayıb. Əsas fakt budur: **Order obyekti var, amma Items bazadan oxunmayıb.**

# Eager: məhsulların lazım olduğunu əvvəlcədən de

**Eager loading** əlaqəli məlumatı əsas sorğu ilə birlikdə tələb etməkdir. Burada bunu Include ilə ifadə edirik:

C# - eager sınağı

```
var query = db.Orders.Include(o => o.Items);
// SQL: 0

var order = await query.SingleAsync(o => o.Id == 42);
// SQL: 1, Order və onun 2 məhsulu artıq gəlib.

Console.WriteLine(order.Items.Count);
// 2; SQL yenə 1-dir.
```

`Include(o => o.Items)` “alacağın sifarişlərin Items əlaqəsini də doldur” tələbidir. Sorğunu yalnız qurur; icranı SingleAsync başladır.

SingleAsync-dən sonrakı vəziyyət belədir:

C# obyektlərinin məntiqi görünüşü

```
order: Id = 42, CustomerName = Aysel
  Items[0]: Cay
  Items[1]: Tort
```

Yəni Items.Count oxunanda məhsulları gətirmək üçün əlavə SQL lazım deyil. Onlar artıq gəlib.

Bu nümunədə bir SQL işləyir. Eager-in əsas mənası **əlaqəni əvvəlcədən sorğuda istəməkdir**. Hər layihədə mütləq bir SQL olacağını düşünmək əvəzinə faktiki əmrlərin sayını yoxla.

**Əgər ekranda sifarişlə məhsullar mütləq göstəriləcəksə**, bu tələbi sorğunun özündə görmək rahatdır.

# Explicit: məhsulları ayrıca əmrlə istə

**Explicit loading** əlaqəni yükləmək üçün açıq şəkildə ayrıca əmr yazmaqdır. Əvvəl Order gəlir, sonra lazım olsa Items gətirilir:

C# - explicit sınağı

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
// SQL: 1; Items hələ gətirilməyib.

await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync();
// SQL: 2; indi Items-də 2 məhsul var.

Console.WriteLine(order.Items.Count); // 2
// SQL yenə 2-dir.
```

LoadAsync “bu konkret order obyektinin Items əlaqəsini yüklə” deyir. EF order.Id-nin 42 olduğunu bildiyi üçün məhsul sorğusuna bu şərti qoyur:

SQL - ikinci əmrin sadələşdirilmiş mənası

```
SELECT Id, OrderId, ProductName
FROM OrderItems
WHERE OrderId = 42;
```

Burada əlavə SQL-i Items.Count başlamadı. **Əlavə SQL-i LoadAsync sətri başladı.** Count oxunanda artıq gəlmiş siyahının sayı alınır.

Məsələn, sifarişi gətirəndən sonra iş qaydasına əsasən məhsulların lazım olub-olmadığını qərarlaşdırırsansa bu qərarın altına LoadAsync yaza bilərsən. Əlaqə artıq yüklənmiş sayılırsa eyni LoadAsync çağırışını təkrarlamaq bu sınaqda yeni SQL yaratmır; bu, məcburi yeniləmə əmri deyil.

# Lazy: əvvəl sifariş gəlir, məhsullar gözləyir

**Lazy loading** artıq alınmış entity-nin hələ yüklənməmiş əlaqəsini, həmin əlaqəyə müraciət ediləndə gətirməkdir. Sifarişin özünü gətirməyi gözlətmirik; bu nümunədə gözləyən iş onun məhsullarını gətirməkdir.

Bazada Order 42 Ayselə aiddir və iki məhsulu var: Cay və Tort. Lazy aktiv, context təzədir və əlaqəni əvvəlcədən istəmirik:

C# - birinci SQL burada işləyir

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
```

Bu sətirdən sonra Order-un Id və CustomerName məlumatları gəlib. **Items-in yüklənməsi isə hələ baş verməyib.**

C# - bu oxunuş ikinci SQL yaratmır

```
Console.WriteLine(order.CustomerName); // Aysel
```

CustomerName sifarişin öz məlumatıdır; birinci nəticə ilə artıq gətirilib. Items isə başqa cədvəldəki əlaqəli məhsullara keçiddir:

C# - ikinci SQL məhz burada işləyir

```
var items = order.Items;
```

Bu sətir “42-nin məhsullarını gətir” işini başladır. Məhsullar gəlib kolleksiyaya yerləşəndən sonra items dəyişəni həmin kolleksiyanı alır.

**“Lazım olanda gətirir” ifadəsində proqramın fikrini təxmin etmək yoxdur. Tətik konkret əməliyyatdır: yüklənməmiş Items xassəsini oxumaq.** Items heç oxunmasa, bu təcrübədə məhsul SQL-i də işləməz.

# Adi xassə oxunuşu SQL-i necə işlədir?

Özümüz yazdığımız adi List avtomatik bazaya qoşulmur. Bunun üçün burada **lazy-loading proxy** mexanizmini seçirik. **Proxy** EF-in Order sinfindən törədib yaratdığı xüsusi obyekt variantıdır. Törəmə sinif başqa sinfin imkanlarını götürüb genişləndirən sinifdir.

Terminal - layihəyə uyğun paket əlavə edilir

```
dotnet add package Microsoft.EntityFrameworkCore.Proxies --version 10.0.9
```

İşlək layihədə paket artıq var. **Paket** layihəyə əlavə edilən hazır kitabxanadır. Context ayarını quran builder üzərində bunu aktivləşdiririk:

C# - layihənin ayarı

```
builder.UseLazyLoadingProxies();
```

C# - Order sinfindəki navigation

```
public virtual ICollection<OrderItem> Items { get; set; }
    = new List<OrderItem>();
```

**Getter** xassə oxunanda işləyən hissədir. `virtual` törəmə sinfin həmin davranışı dəyişməsinə imkan verir. **Override** törəmə sinifdə həmin davranışın yenidən təyin edilməsidir. Proxy Items getter-inə müdaxilə edib əlaqənin yüklənməsini başladır, sonra kolleksiyanı qaytarır.

Bu proxy variantında Order sinfi törədilə bilən olmalıdır; nümunədə public-dır və sealed deyil. **Sealed** sinifdən törəmə yaratmağı qadağan edir. **Tək virtual yazmaq kifayət deyil:** paket, ayar və EF-in yaratdığı uyğun proxy obyekt də lazımdır.

EF-də proxy-siz lazy mexanizmi də mövcuddur. Buradakı virtual tələbi məhz seçdiyimiz proxy üsuluna aiddir.

# new Order ilə EF-in verdiyi Order eyni davranmır

C# - adi obyekt; bazaya sorğu yoxdur

```
var plain = new Order { Id = 42 };
Console.WriteLine(plain.GetType().Name); // Order
Console.WriteLine(plain.Items.Count);   // 0
```

Id-ni 42 yazmağımız bazadakı Order 42-ni gətirmədi. Sadəcə yeni C# obyektinə nömrə verdik. Onun Items-i sinifdə yaratdığımız boş List-dir. **Bu modeldə adi new Order() proxy yaratmır və təkcə Items-i oxumaq bazaya getmir.**

C# - proxy aktiv ayarlarla bazadan oxuyuruq

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
Console.WriteLine(order.GetType().Name); // OrderProxy
```

`GetType()` obyektin həqiqi işləmə vaxtı tipini göstərir. Bu sınaqda EF-in yaratdığı tipin adı OrderProxy-dir. Bu tipin adı kitabxana detalıdır; kodun işləməsini həmin ada bağlamırıq.

OrderProxy Order-dan törədiyi üçün onu Order dəyişənində saxlaya bilirik. Sən kodda `Order` görürsən, amma xassə oxunuşunda xüsusi davranışa malik obyektlə işləyirsən.

Məntiqi yol - EF-in mənbə kodu deyil

```
order.Items oxunur
-> proxy yüklənməmiş əlaqəni aşkar edir
-> EF OrderId = 42 məhsullarını oxuyur
-> obyektlər Items-ə yerləşdirilir
-> getter hazır kolleksiyanı qaytarır
```

Paket əlavə etmək əvvəldən yaratdığın bütün adi Order obyektlərini öz-özünə proxy-yə çevirmir.

# Items və Count-u iki ayrı addımda gör

C# - lazy aktivdir, context təzədir

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42);
// SQL: 1; Items hələ yüklənməyib.

var items = order.Items;
// Getter məhsulları gətirdi; SQL: 2.

var count = items.Count;
// Hazır kolleksiyanın sayıdır: 2; SQL yenə 2.

var again = order.Items;
// Əlaqə artıq yüklənib; SQL yenə 2.
```

`order.Items.Count` bu nümunədə əvvəl `order.Items`-i oxuyur, sonra alınmış kolleksiyanın Count-unu oxuyur. **SQL COUNT əmri işləmir. Məhsul obyektləri gətirilir, sonra onların sayı alınır.**

SQL - ikinci əmrin sadələşdirilmiş mənası

```
SELECT Id, OrderId, ProductName
FROM OrderItems
WHERE OrderId = 42;
```

İlk getter oxunuşu bitəndə kolleksiyada Cay və Tort obyektləri var, əlaqə yüklənmiş sayılır. Növbəti adi oxunuş həmin kolleksiyanı qaytarır; avtomatik olaraq hər dəfə bazanı yeniləmir.

Bu davranış yüklənməmiş əlaqə üçündür. Əgər əvvəl Include və ya LoadAsync ilə əlaqəni gətirmiş olsaydıq, getter bu təcrübədə yenidən sorğu işlətməzdi.

Context istifadəyə açıq qalmalıdır; bağlanmış context ilə hələ yüklənməmiş əlaqəni gətirməyə arxalanma. Əsas sorğudakı await getter-i asinxron etmir: bu proxy getter-i sinxron yükləyir. Asinxron yükləmə anını özün göstərmək üçün explicit LoadAsync istifadə edə bilərsən.

# İşlədərək gör: hansı addım SQL sayını artırır?

İşlək layihə qovluğunda `dotnet run -- lazy-steps` əmrini ver. Əvvəl adi new obyekt yoxlanır, sonra təzə context-dən Order 42 alınır. Çıxışın əsas hissəsi belədir:

Yoxlanmış nəticə

```
New tip: Order
New Items: 0
New SQL: 0
EF tip: OrderProxy
1) SQL: 1
1) Yuklenib: False
2) Ad: Aysel
2) SQL: 1
3) SQL: 2
3) Yuklenib: True
4) Items: 2
4) SQL: 2
5) Eyni kolleksiya: True
5) SQL: 2
```

**1:** əsas Order alınıb; Items yüklənməyib. Yüklənmə vəziyyətini Items-ə toxunmadan `db.Entry(order).Collection(o => o.Items).IsLoaded` ilə yoxlayırıq.

**2:** CustomerName oxunub; bu dəyər artıq vardı, əlavə SQL yoxdur.

**3:** `var items = order.Items;` yazılıb. Təkcə bu getter oxunuşu ikinci SQL-i başladıb; əlaqə artıq yüklənib.

**4:** `items.Count` 2-ni qaytarıb; əlavə SQL yoxdur.

**5:** Items yenidən oxunub. `ReferenceEquals` iki istinadın eyni obyektə gedib-getmədiyini yoxlayır. Burada eyni kolleksiya qaytarılıb və SQL sayı artmayıb. Çıxışın ardınca faktiki iki SQL qeydi də göstərilir.

# Üç variantı bir zaman xəttində müqayisə et

Hər sınaqda eyni 42 nömrəli sifarişi alırıq. Ayarlar əvvəlki səhifələrdəki kimidir və başlanğıcda əlaqələr yüklənməyib.

| Addım | Eager | Explicit | Lazy |
|---|---|---|---|
| Sorğunu qururuq | Include var; SQL 0 | Include yox; SQL 0 | Include yox; SQL 0 |
| SingleAsync bitir | Order + Items hazır; SQL 1 | Order hazır; SQL 1 | Order hazır; SQL 1 |
| Sonrakı addım | Items.Count | LoadAsync, sonra Count | Items.Count |
| Əlavə SQL-i başladan | Əlavə SQL yoxdur | LoadAsync | İlk Items getter-i |
| Yekun Items / SQL | 2 / 1 | 2 / 2 | 2 / 2 |

**Explicit və lazy-nin burada SQL sayı eynidir, amma nəzarət nöqtəsi fərqlidir.** Explicit-də yükləmə əmrini kodda görürsən. Lazy-də uyğun xassə oxunuşu bu işi başladır.

**Eager və deferred bir yerdə ola bilər.** Include yazıb sonra ToListAsync gözləmək bunun nümunəsidir. Eager əlaqənin tələb olunma qaydasıdır, deferred əsas sorğunun nəticəsinin nə vaxt oxunmasıdır.

**IEnumerable və lazy də eyni anlayış deyil.** IEnumerable elementləri oxumaq müqaviləsidir. Lazy loading isə entity əlaqəsinə müraciətdə ayrıca məlumat gətirmək mexanizmidir.

Özün yoxla: `var q = db.Orders.Include(o => o.Items);` sətrində SQL sayı neçədir? **0.** Items-in lazım olduğunu dedik, amma nəticəni hələ istəmədik.

# Bu dörd mövzunu ayırd etmək üçün

**Deferred execution:** “Əsas sorğunu nə vaxt işlədirəm?” Sorğunu hazırlayıb nəticəsini ToList, foreach və ya başqa nəticə istəyən əməliyyata qədər gözlətməkdir.

**IQueryable:** “Sorğuya hansı şərtləri əlavə edə bilirəm?” EF provider-i uyğun ifadələri SQL-ə çevirə bilir. Təkcə IQueryable adı mənbənin mütləq SQL bazası olduğunu bildirmir.

**IEnumerable:** “Elementləri necə oxuyuram?” Bu müqavilə hazır List-i də, hələ işlənməmiş mənbəni də ifadə edə bilər. Məlumatın artıq gəldiyini yalnız tipin adından bilmək olmur.

**Loading:** “Əlaqəli Items məlumatını hansı anda gətirirəm?” Eager-də sorğuda Include, explicit-də ayrıca LoadAsync, aktiv lazy-də isə yüklənməmiş navigation-un ilk oxunuşu bunu müəyyən edir.

Hər kodda əvvəl mənbəni və görünən tipi müəyyən et. Sonra SQL-i başladan sətri tap. Axırda Order-un və Items-in hansı anda hazır olduğunu de.

Rəsmi istinadlar:

- IQueryable, AsEnumerable və deferred execution.
- Eager loading, explicit loading və lazy loading.

# 4 Function və stored procedure nə ilə fərqlənir

Kafedə iki ayrı ehtiyacımız var. Birincisi, məbləğdən endirimi çıxıb cavab almaqdır. İkincisi, sifarişin vəziyyətini bazada dəyişməkdir. Birinci işin nəticəsi hesablanmış dəyərdir; ikinci iş bazada əməliyyatdır.

SQL Server-də **function** bir dəyər və ya cədvəl nəticəsini sorğunun içində istifadə etmək üçün verir. **Stored procedure** isə adla saxlanmış əmrləri icra edir. Hər ikisi bazada saxlanan təkrar istifadə olunan kod ola bilər.

**SQL** cədvəllərlə işləmək üçün dildir. `SELECT` məlumatı oxuyur, `INSERT` yeni sətir əlavə edir, `UPDATE` mövcud sətri dəyişir, `DELETE` sətri silir. SQL Server-də əlavə imkanları olan bu dil variantı **T-SQL** adlanır.

İki ehtiyac

```
"100-den 15 cix, cavabi ver." -> Function ucun numune.

"42-ci sifarisi tesdiqle."   -> Procedure ucun numune.
```

Bu dərsdəki function məhdudiyyətləri **SQL Server T-SQL funksiyalarına** aiddir. PostgreSQL kimi başqa bazalarda eyni qaydaları yoxlamadan tətbiq etmək olmaz.

# Scalar function bir dəyər qaytarır

Endirimdən sonrakı məbləği hesablayaq:

T-SQL

```
CREATE OR ALTER FUNCTION dbo.NetAmount
(
    @Gross decimal(12,2),
    @Discount decimal(12,2)
)
RETURNS decimal(12,2)
AS
BEGIN
    RETURN @Gross - @Discount;
END;
GO

SELECT dbo.NetAmount(100, 15) AS Net;
```

Nəticə **85.00** olur. Tək nəticə dəyərinə **scalar** deyilir. `@Gross` və `@Discount` giriş parametrləridir: çağırış zamanı verilən dəyərlər. `decimal(12,2)` onluq tipdir; cəmi 12 rəqəm, onlardan 2-si vergüldən sonra ola bilər.

`RETURNS` nəticənin tipini, `RETURN` nəticənin özünü bildirir. `AS Net` cavab sütununa Net adını verir. dbo obyektin aid olduğu schema-dır. **Schema** baza obyektlərini ad altında qruplaşdırır.

`GO` SQL Server-i idarə etmək üçün istifadə olunan alətlərdə əmrlər qrupunu ayırır. Serverin adi T-SQL əmri deyil. Tətbiqdən göndərərkən həmin qruplar ayrı göndərilir.

**Əsas fərq:** function nəticəsini başqa SELECT ifadəsinin içində istifadə edə bilirik.

# Procedure əmrləri adla icra edir

Orders cədvəlində `Id`, `CustomerId` və `Total` sütunlarının olduğunu qəbul edək. Müştərinin sifarişlərini gətirən procedure yazaq:

T-SQL

```
CREATE OR ALTER PROCEDURE dbo.GetOrders
    @CustomerId int
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;
END;
GO

EXEC dbo.GetOrders @CustomerId = 7;
```

`EXEC` procedure-u çağırır. `int` tam ədəd tipidir. `FROM` məlumatın hansı cədvəldən alınacağını, `WHERE` isə hansı sətirlərin seçiləcəyini bildirir. Bu procedure müştəri 7-nin sifariş cədvəlini qaytarır.

Qayıdan sətir və sütunlar toplusuna **result set** deyilir. Deməli, “procedure heç nə qaytarmır” düzgün izah deyil.

`SET NOCOUNT ON` təsirlənən sətir sayı barədə əlavə mesajları azaldır. SELECT nəticəsini gizlətmir. Onu hər sorğunu böyük faizlə sürətləndirən qayda kimi əzbərləmə.

Procedure SELECT edə bildiyi kimi INSERT, UPDATE və DELETE də edə bilər. SQL Server-də istifadəçinin yazdığı T-SQL function isə bazanın xarici vəziyyətini dəyişən belə iş vahidi üçün nəzərdə tutulmur.

# Procedure nəticəni üç ayrı yolla verə bilər

Əvvəlki səhifədə sifariş cədvəlini SELECT ilə aldıq. Procedure ayrıca çıxış parametrini də doldura bilər:

T-SQL

```
CREATE OR ALTER PROCEDURE dbo.CountOrders
    @CustomerId int,
    @Count int OUTPUT
AS
BEGIN
    SELECT @Count = COUNT(*)
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;

    RETURN 0;
END;
GO

DECLARE @N int, @Status int;
EXEC @Status = dbo.CountOrders
    @CustomerId = 7, @Count = @N OUTPUT;
SELECT @N AS OrderCount, @Status AS Status;
```

`COUNT(*)` sətirləri sayır. `OUTPUT` procedure-un doldurduğu dəyəri çağırana çıxarır. `@N` say olur. `RETURN 0` isə bu nümunədə uğuru bildirən tam ədəd statusudur; `@Status` onu alır. `DECLARE` dəyişən elan edir.

Üç yolu ayır: **SELECT nəticə cədvəli verir; OUTPUT parametr dəyəri verir; procedure RETURN-u tam ədəd status verir.** Procedure RETURN-u function-un məbləğ qaytaran RETURN-u ilə eyniləşdirmə.

# Table valued function cədvəl kimi istifadə olunur

Function yalnız bir rəqəm qaytarmağa məcbur deyil. **Table-valued function**, qısa **TVF**, cədvəl qaytarır. Onu başqa sorğunun məlumat mənbəyi kimi işlətmək olur.

T-SQL

```
CREATE OR ALTER FUNCTION dbo.CustomerOrders
(
    @CustomerId int
)
RETURNS TABLE
AS
RETURN
(
    SELECT Id, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId
);
GO

SELECT Id, Total
FROM dbo.CustomerOrders(7)
WHERE Total > 20;
```

Function əvvəl müştərinin sifarişlərini müəyyən edir. Xarici sorğu isə nəticəni `Total > 20` ilə daraldır. Belə əlaqələndirilə bilmə function-un vacib üstünlüyüdür. Procedure normal SELECT-də bu cür birbaşa FROM mənbəyi kimi yazılmır.

Bir SELECT ilə yazılan bu formaya **inline TVF** deyilir. Baza onu əsas sorğu ilə birlikdə qiymətləndirə bilər.

**Multi-statement TVF** isə əvvəl nəticə cədvəlini dəyişən kimi yaradır, bir neçə əmrlə doldurur və qaytarır. Daha çox addım yazmaq mümkündür, amma planın seçilməsi çətinləşə bilər. Daha mürəkkəb forma avtomatik daha sürətli forma deyil.

# Birlikdə tamamlanmalı dəyişikliklər transaction istəyir

Sifariş təsdiqlənəndə həm Orders vəziyyəti dəyişsin, həm məhsul ehtiyatı azalsın. Birinci dəyişiklik yazılıb ikinci uğursuz olsa, məlumat yarımçıq qalar.

**Transaction** birlikdə təsdiqlənməli baza işlərinin sərhədidir. `COMMIT` təsdiqləyir, `ROLLBACK` həmin transaction-un dəyişikliklərini geri alır. Bu birlikdəlik xüsusiyyətinə **atomiklik** deyilir.

Məntiqi axın

```
Transaction basla.
    Sifarisi tesdiqle.
    Ehtiyati azalt.

Her ikisi ugurludur -> COMMIT.
Xeta var           -> ROLLBACK.
```

Procedure transaction və xəta idarəetmə məntiqi daşıya bilər. T-SQL UDF, yəni istifadəçinin yaratdığı SQL Server function-u, bunun üçün uyğun iş vahidi deyil.

`TRY/CATCH` xətalı icranı tutmaq üçün bloklardır. `THROW` xətanı çağırana ötürür. Əsas fikir xəta gizlədib uğur demək əvəzinə nəticəni düzgün bildirməkdir.

Transaction bazadan kənara göndərilmiş email-i geri ala bilmir. Eyni qaydada başqa transaction-un içində çağırılan procedure-un daxili COMMIT-i müstəqil xarici təsdiq deyil. Transaction-un kim tərəfindən açıldığını bilmək lazımdır.

**Seçim:** bir-birinə bağlı baza dəyişikliklərini təşkil etmək üçün procedure və ya tətbiqin idarə etdiyi transaction düşün. Function-u nəticə ifadəsi kimi seç.

# Performans və təhlükəsizlik barədə düzgün nəticə

Baza sorğunu icra etmək üçün yol seçir. Həmin yol **execution plan**, icra planıdır. Planın sonradan təkrar istifadə üçün saxlanmasına **plan cache** deyilir.

Procedure çox vaxt planı təkrar istifadə edə bilər. Amma plan yenidən qurula bilər və parametrli adi SQL də təkrar istifadədən yararlana bilər. Ona görə “procedure həmişə əvvəlcədən hazırdır və daha sürətlidir” doğru nəticə deyil.

Eyni function-u çox sətir üçün ayrı hesablatmaq da baha ola bilər. Bəzi SQL Server versiya və şərtlərində function əsas plana açıla bilər; buna **inlining** deyilir. Sürət barədə son qərar real plana və ölçməyə əsaslanır.

**SQL injection** istifadəçi mətninin məlumat əvəzinə SQL əmrinə çevrilməsidir. Procedure daxilində istifadəçi mətnini SQL-ə birləşdirmək bu riski yenə yarada bilər. Dəyərləri ayrıca parametr kimi ötürmək lazımdır.

### Özünü yoxla

**Hesablanmış məbləği SELECT sütunu kimi istəyirsən:** scalar function və ya sadə ifadə düşün.

**Sifariş nəticəsini başqa cədvəllə birləşdirmək istəyirsən:** inline TVF uyğun ola bilər.

**Sifarişi, ehtiyatı və audit qeydini dəyişmək istəyirsən:** procedure və transaction sərhədini düşün. Audit baş vermiş əməliyyatın qeydidir.

# 5 İndeks məlumatı tapmağı necə dəyişir

Kafedə on minlərlə sifariş var. “Müştəri 7-nin sifarişlərini gətir” deyirik. Bazada uyğun axtarış quruluşu yoxdursa çoxlu sifariş səhifəsinə baxmaq lazım ola bilər.

Kitabda bir mövzunu axtarmağı düşün. Bütün səhifələri bir-bir oxumaq olar. Yaxud arxadakı göstəricidən mövzunun yerini tapıb həmin səhifəyə keçmək olar.

Baza **indeksi** həmin göstəriciyə bənzəyən ayrıca axtarış quruluşudur. Axtarılan dəyərlə məlumatın yerini əlaqələndirir.

İndeksin verdiyi fikir

```
Indeks yoxdur:
    Coxlu melumat sehifesine bax, uygun setirleri sec.

Uygun indeks var:
    Axtaris qurulusunda lazim olan hisseye cat.
    Uygun melumati oxu.
```

İndeksin məqsədi məlumatın özünü dəyişmək deyil, **uyğun sorğuda görülən oxuma işini azaltmaqdır**.

Amma kitabın yarısını oxumaq istəyirsənsə hər səhifəyə ayrıca göstəricidən getmək sərfəli olmaya bilər. Bazada da çoxlu nəticə qaytaran sorğu üçün başqa yol daha ucuz ola bilər.

# Sütuna indeks verəndə nə yaranır

Bu SQL Server cədvəlini düşün:

| Id | CustomerId | Total |
|---|---|---|
| 42 | 7 | 30 |
| 43 | 7 | 10 |
| 44 | 9 | 20 |

T-SQL

```
CREATE INDEX IX_Orders_Customer
ON dbo.Orders(CustomerId);
```

Bu əmr `CustomerId` üzrə indeks yaradır. Sütunun içindəki 7 və 9 dəyərləri dəyişmir. Əlavə quruluş təxminən “müştəri 7 üçün bu sətirlərə, 9 üçün o sətrə bax” imkanı verir.

Sadələşdirilmiş xəritə

```
CustomerId 7 -> Order 42, Order 43
CustomerId 9 -> Order 44
```

Bu xəritə axtarış fikrini göstərir; real daxili quruluş bir qədər sonra izah olunur.

İndi `WHERE CustomerId = 7` şərti ilə sorğu gələndə baza bu indeksdən istifadə etməyi qiymətləndirə bilər. `WHERE` sətirləri seçən şərtdir. İndeksin mövcudluğu mütləq həmin indeksin seçilməsi demək deyil.

**Yadda saxla:** sütuna indeks vermək, o sütuna əlavə axtarış quruluşu yaratmaqdır. Hər sorğuya sürət zəmanəti vermək deyil.

# B plus tree böyük siyahını hissələrə bölür

Adi SQL Server rowstore indeksləri **B+ tree** adlı balanslı ağac quruluşundan istifadə edir. Rowstore məlumatın sətir əsaslı saxlanmasıdır.

Ağacda yuxarıdan aşağı hər dəfə uyğun aralığı seçirik. Hər sətri əvvəldən yoxlamaq əvəzinə uyğun hissəyə yaxınlaşırıq:

Sadələşdirilmiş quruluş

```
Baslangic: [1..100] [101..200] [201..300]
                       |
                       v
                [101..150] [151..200]
                                |
                                v
                       [171, 172, 173, 174, ...]
```

Başlanğıc hissə **root**, aralıq hissə **branch**, son səviyyə **leaf** adlanır. Hər real qovşaqda çoxlu açar ola bilər. Bu, hər qovşağı yalnız iki hissəyə bölünən sadə binary ağac deyil.

**Page**, səhifə, bazanın məlumat saxladığı blokdur. SQL Server-in adi rowstore səhifəsi 8 KB-dır və içində bir neçə sətir ola bilər. Buna görə bir milyon sətirə baxmaq mütləq bir milyon disk oxunuşu demək deyil.

**O(log n)** böyümə barədə nəzəri ifadədir: məlumat artdıqca ağacda yolu tapmaq işi sətirlərin hamısını gəzmək qədər artmır. Bu ifadə dəqiq saniyə və disk oxunuş sayı vermir. Tapıldıqdan sonra nəticələrin özünü oxumağın da xərci vardır.

# Clustered və nonclustered fərqi

İndi iki quruluşu ayıraq. Bu səhifə SQL Server-in adi sətir əsaslı indekslərinə aiddir.

**Clustered index:** ağacın yarpaq səviyyəsində cədvəlin məlumat sətirləri yerləşir. Yəni son səviyyəyə çatanda məlumatın özünə çatırıq. Bu səbəbdən cədvəldə bir clustered rowstore index olur.

**Nonclustered index:** ayrıca ağacdır. Yarpaqda indeks məlumatı və əsas sətri tapmağa kömək edən yer göstəricisi saxlanır.

Nonclustered indeksdən istifadə nümunəsi

```
CustomerId 7-ni indeksde tap.
    |
    v
Uyqun sifarisin yerini elde et.
    |
    v
Lazim olan diger sutunlari esas setirden oxu.
```

Yer göstəricisinə **locator** deyilir. Cədvəldə clustered index varsa nonclustered indeks clustered açardan istifadə edir. Clustered index olmayan SQL Server cədvəli **heap** adlanır; onda locator sətrin yerini başqa qayda ilə göstərir.

İndeksdən sonra çatışmayan məlumat üçün əsas sətrə getməyə **lookup** deyilir. Az nəticədə ucuz, çox nəticədə baha ola bilər.

Clustered index açar sırasına görə məntiqi quruluş yaradır. Bütün səhifələrin diskdə daim yan-yana olması və SELECT nəticəsinin avtomatik sıralı gəlməsi zəmanəti deyil. Cavab sırası üçün `ORDER BY` yazılır.

# Primary key və unique indeks eyni sualı cavablandırmır

**Primary key**, əsas açar, cədvəldə hər sətri fərqləndirən qaydadır. Onun dəyəri təkrarlanmamalı və NULL olmamalıdır. NULL burada dəyərin olmadığını və ya naməlum olduğunu bildirən SQL işarəsidir.

**Unique index** müəyyən açarın təkrarlanmasına icazə vermir. Məsələn, sistemin qaydası email-in təkrarlanmamasıdırsa:

T-SQL

```
CREATE UNIQUE INDEX UX_Customers_Email
ON dbo.Customers(Email);
```

İki paralel əməliyyat eyni email-i yazmağa çalışanda unikal qaydanı baza özü qoruyur. Tətbiqdə “əvvəl yoxla, sonra yaz” demək təkbaşına bu zəmanəti vermir: iki əməliyyat da eyni anda “yoxdur” cavabı ala bilər.

Primary key məntiqi məlumat qaydasıdır. Clustered isə məlumatın saxlanma quruluşudur. SQL Server primary key üçün dəstək indeksi yaradır, amma primary key-in mütləq clustered olması tələb deyil.

**Foreign key**, xarici açar, başqa cədvəldəki uyğun açara istinad qaydasıdır. SQL Server-də belə qaydanın mövcudluğu həmin sütun üçün mütləq avtomatik əlavə indeks yaradıldığı demək deyil.

**Nəticə:** “nəyi unikal saxlamalıyam?”, “hansı əlaqə etibarlıdır?” və “sorğunu necə sürətləndirim?” əlaqəli, amma ayrı suallardır.

# İki sütunlu indeksdə sıra niyə dəyişir

Telefon kitabçasında əvvəl soyad, sonra ad sırası olduğunu düşün. Soyadı biləndə uyğun hissəyə rahat gedirsən. Tək adı biləndə eyni ad müxtəlif soyadların arasında yayılır.

**Composite index** bir neçə sütunun birlikdə açar olduğu indeksdir. `(CustomerId, CreatedAt)` əvvəl müştəriyə, həmin müştərinin içində tarixə görə qurulur.

İndeksin məntiqi sırası

```
Musteri 7
    2026-10-20
    2026-10-21
Musteri 9
    2026-10-19
    2026-10-23
```

Müştəri 7-nin müəyyən tarixdən sonrakı sifarişləri bu quruluşa uyğundur. Yalnız tarixlə axtarışda isə həmin tarixlər müxtəlif müştəri hissələrində yayılır.

T-SQL

```
CREATE INDEX IX_Orders_Customer_Date
ON dbo.Orders(CustomerId, CreatedAt);
```

“Ən çox fərqli dəyəri olan sütun həmişə əvvəl olmalıdır” qaydasını əzbərləmə. Əvvəl hansı sorğunun hansı bərabərlik və aralıq şərtlərindən istifadə etdiyini müəyyən et. Sonra sıralamanı və başqa sorğuları nəzərə al.

**Yadda saxla:** sütunların siyahısı qədər həmin siyahının sırası da indeksin hansı sorğuya rahat yol verdiyini dəyişir.

# INCLUDE əsas sətrə əlavə gedişi azalda bilər

Sorğu müştəri və tarixlə axtarır, amma cavabda Total məbləğini də istəyir. İndeksdə Total yoxdursa baza onu əsas sətirdən ayrıca oxuya bilər.

T-SQL

```
CREATE INDEX IX_Orders_Customer_Date
ON dbo.Orders(CustomerId, CreatedAt)
INCLUDE(Total);
```

`INCLUDE(Total)` məbləği də indeksin yarpaq səviyyəsində saxlayır. Amma Total axtarış açarlarının sırasına əlavə edilmir.

Bu sorğunun lazım olan məlumatı

```
Filtr:  CustomerId ve CreatedAt.
Cavab:  CreatedAt ve Total.

Hamisi bu indeksde tapila biler.
```

Bir sorğunun ehtiyac duyduğu bütün məlumat indeksdədirsə həmin sorğu üçün ona **covering index** deyilir. Covering “əhatə edən” deməkdir. Eyni indeks başqa, daha çox sütun istəyən sorğunu əhatə etməyə bilər.

Hər sütunu INCLUDE etmək də pulsuz deyil. İndeks böyüyür, daha çox yer tutur və həmin məlumat dəyişəndə əlavə iş yaradır.

**Nəticə:** açar sütunları axtarış və sıra üçün, INCLUDE sütunları isə nəticədə lazım olan əlavə məlumat üçün seçilir.

# İndeks niyə bəzən seçilmir

Hansı yolu seçən hissəyə **optimizer** deyilir. O, sorğunu yerinə yetirmək üçün mümkün yolların xərcini qiymətləndirir.

**Index seek** uyğun açar aralığından başlamaqdır. **Index scan** indeksin böyük hissəsini və ya hamısını gəzməkdir. **Table scan** cədvəl məlumatını gəzməkdir.

Milyon sifarişin yalnız 3-ü lazımdırsa dar indeks axtarışı sərfəli ola bilər. Sifarişlərin 90 faizi lazımdırsa hər sətrə ayrıca lookup etməkdənsə geniş oxuma daha ucuz ola bilər. Seek həmişə yaxşı, scan həmişə pis deyil.

**Selectivity** şərtin məlumatın nə qədərini seçməsini ifadə edir. **Cardinality** kontekstdən asılı olaraq fərqli dəyər sayını və ya nəticə sətir sayını bildirə bilər.

Məsələn, iki statusdan yalnız biri cəmi 100 sətrə aiddirsə, həmin nadir hissə üçün indeks faydalı ola bilər. “Bool və az variantlı sütuna indeks həmişə mənasızdır” düzgün deyil.

**Filtered index** yalnız müəyyən şərtə uyğun sətirləri saxlayan SQL Server indeksidir. PostgreSQL-də oxşar anlayış partial index adlanır.

Optimizer-in dəyərlərin paylanmasını təxmin etməsinə kömək edən məlumata **statistics**, statistika deyilir. Səhv təxmin uyğun olmayan plan seçiminə səbəb ola bilər.

# Oxuma qazancının yazma xərci var

İndeks cədvəllə uyğun qalmalıdır. Buna görə əlavə iş yaranır:

| Dəyişiklik | İndeksdə görülən iş |
| --- | --- |
| INSERT | Yeni sətrə uyğun indeks məlumatı əlavə edilir. |
| İndeksdə saxlanan dəyərə UPDATE | Müvafiq indeks məlumatı yenilənir. |
| DELETE | Sətrin indeks məlumatı da silinmə baxımına düşür. |

İndeks ayrıca disk sahəsi tutur. İstifadə olunan səhifələr yaddaşda yer tutur. Dəyişikliklərin bərpa üçün yazıldığı **transaction log** yükü də arta bilər.

Sütunun üzərində hesablama aparmaq axtarışı çətinləşdirə bilər:

T-SQL şərtləri

```
-- Ilk forma her tarixden il hesablayir:
WHERE YEAR(CreatedAt) = 2026

-- Ikinci forma tarix araligini birbasa gosterir:
WHERE CreatedAt >= '20260101'
  AND CreatedAt <  '20270101'
```

YEAR tarixdən ili çıxarır. İkinci forma uyğun indeks üçün daha rahat aralıq yarada bilər. İndeksdə səmərəli axtarış şərti kimi istifadə oluna bilən yazılışa **SARGable** deyilir. Bəzi hesablanmış sütun indeksləri kimi istisnalar da var; “funksiya varsa heç bir indeks işləməz” hökmü də doğru deyil.

# PostgreSQL-də xüsusi indekslər nə üçündür

Bu səhifədəki növlər PostgreSQL üçündür. Onların adlarını SQL Server-in bütün cədvəllərinə eyni qayda ilə tətbiq etmək olmaz.

**B-tree** sıralana bilən dəyərlərdə bərabərlik və aralıq axtarışı üçün ümumi seçimdir. **Hash** dəyərdən hesablanan kodla sadə bərabərlik axtarışına xidmət edir; adi tarix aralığı sıralaması vermir.

**GIN** dəyərin içindəki hissədən onu saxlayan sətirlərə keçid yaradır. Məsələn, etiket siyahıları varsa:

GIN fikri

```
"cay" etiketi  -> 1, 4, 9 nomreli mehsullar.
"sirin" etiketi -> 2, 4 nomreli mehsullar.
```

**GiST** müxtəlif tiplərə uyğun axtarış qurmağa imkan verən çərçivədir. Həndəsi məlumatda bölgələrdən namizəd yerləri tapmaq nümunədir. **SP-GiST** məkanı hissələrə ayıran başqa quruluşları dəstəkləyir. Hansı müqayisələrin işləməsini konkret tip üçün operator qaydaları müəyyən edir.

**BRIN** hər sətir üçün ayrıca yer saxlamaq əvəzinə səhifə aralıqlarının xülasəsini saxlayır. Tarixlər fiziki sıra ilə yaxın gedirsə bəzi aralıqların sorğuya uyğun gəlmədiyini tez müəyyən edə bilər. Namizəd aralıqdakı real sətirlər yenidən yoxlanır.

**Nəticə:** indeks növünü məlumatın və axtarışın formasına görə seçirik. Bütün məlumat növlərinə bir indeks eyni dərəcədə uyğun deyil.

# Düzgün indeks qərarını necə verək

Kafedə əsas sorğumuz belə olsun: müştərinin müəyyən tarixdən sonrakı sifarişləri, tarix və məbləğlə birlikdə göstərilsin.

T-SQL nümunəsi

```
SELECT CreatedAt, Total
FROM dbo.Orders
WHERE CustomerId = @CustomerId
  AND CreatedAt >= @From;
```

`@CustomerId` müştəri nömrəsi, `@From` başlanğıc tarix parametridir. `AND` hər iki şərtin doğru olmasını tələb edir. Bu sorğu üçün `(CustomerId, CreatedAt) INCLUDE(Total)` məntiqli namizəddir. Son qərar ölçmə ilə verilir.

SQL Server-də **execution plan** sorğunun hansı addımlarla icra olunduğunu göstərir. `SET STATISTICS IO ON` oxunan səhifələr barədə məlumat verir. **Logical read** səhifəyə yaddaş üzərindən müraciətdir. **Physical read** səhifəni diskdən gətirmək ehtiyacıdır; səhifə artıq yaddaşdadırsa yenidən disk oxunuşu lazım olmaya bilər.

PostgreSQL-də `EXPLAIN` planı, `EXPLAIN ANALYZE` isə sorğunu həqiqətən işlədərək icra məlumatını göstərir. Yazma sorğusunda ANALYZE də yazmanı icra edir.

### Özünü yoxla

İndeks qoyandan sonra yalnız “seek göründü” demə. **Oxunan səhifə azaldımı, cavab vaxtı yaxşılaşdımı, əlavə yazma xərci qəbul edilirmi?** Bunları müqayisə et. Sabit “100 dəfə sürətlənəcək” vədi vermək olmaz.

# 6 Fraqmentasiya məlumatın yayılmasıdır

İndeks məlumatı səhifələrdə saxlayır. Vaxt keçdikcə yeni sətirlər əlavə edilir, bəzisi böyüyür, bəzisi silinir. Səhifələrin sırası və doluluğu dəyişə bilər.

Fraqmentasiya barədə düşünəndə iki ayrı sualı ayır:

- Açar sırası ilə getdiyim səhifələr faylda hansı ardıcıllıqla yerləşir?
- Eyni məlumat neçə səhifəyə yayılıb və bu səhifələr nə qədər doludur?

Eyni məlumat üçün iki fərqli vəziyyət

```
Yaxsi doluluq:
    [dolu sehife] [dolu sehife]

Asagi doluluq:
    [yari dolu] [yari dolu] [yari dolu] [yari dolu]
```

İkinci vəziyyətdə oxunacaq və yaddaşda saxlanacaq səhifə sayı arta bilər. Bu, məlumatın yanlış yazıldığı və ya itdiyi demək deyil.

**Fraqmentasiya performans mövzusudur. Corruption isə məlumat quruluşunun korlanmasıdır.** Bunları qarışdırma.

Bu dərsin əsas hissəsi SQL Server rowstore indekslərinə aiddir. Onların səhifələri və B+ tree məntiqi indeks dərsində izah edildi.

# Xarici və daxili fraqmentasiyanı ayıraq

Açar aralığına görə səhifələr A, B, C sırası ilə oxunmalıdır. Amma həmin səhifələrin fayldakı nömrələri belə ola bilər:

Məntiqi sıra

```
A: acarlar 1..20    -> faylda sehife 10
B: acarlar 21..40   -> faylda sehife 90
C: acarlar 41..60   -> faylda sehife 11

Oxuma yolu: 10 -> 90 -> 11
```

Açar sırası ilə fiziki səhifə ardıcıllığının uyğun gəlməməsi **logical fragmentation** adlanır. Çox vaxt buna xarici fraqmentasiya deyilir. Böyük aralıqları oxuyanda ardıcıl gətirmənin səmərəsinə təsir edə bilər.

**Page density** səhifənin doluluğudur. Səhifə daxilində çox boş sahə qalmasına daxili fraqmentasiya kimi baxılır. Eyni məlumat daha çox səhifə tutursa yaddaş və oxuma xərci arta bilər.

Əməliyyat sisteminin baza faylını disk blokları arasında necə yerləşdirməsi isə başqa səviyyədir. **File system fragmentation** buna aiddir. SQL indeksini yenidən qurmaqla ümumi disk defraqmentasiyası eyni əməliyyat deyil.

**Yadda saxla:** sıra problemi ilə boşluq problemini ayrı ölç. Eyni “fraqmentasiya” sözü onların təsirini eyniləşdirmir.

# Page split necə yaranır

Bir səhifədə 10, 20, 30, 40 açarlı sətirlər saxlanır. Səhifədə yeni sətir üçün yer qalmayıb. İndi 25 açarlı sətir əlavə etmək istəyirik.

Baza əlavə səhifə ayırıb sətirlərin bir hissəsini ora köçürə bilər. Bu bölünməyə **page split** deyilir.

Bölünmədən əvvəl

```
Sehife A: [10, 20, 30, 40]  Yer yoxdur.
```

```
Bölünmədən sonra sadələşdirilmiş görünüş
Sehife A: [10, 20]
Sehife B: [25, 30, 40]
```

Yeni səhifə faylın başqa yerində yerləşə bilər. İki səhifədə əlavə boşluq qala bilər. Bölünmə əlavə yazma və log işi yaradır. Silinmələr və böyüyən sətirlər də səhifə doluluğunu dəyişə bilər.

**Fill factor** indeks qurularkən yarpaq səhifələrini hansı hədəf doluluqla yaratmağı bildirir. Məsələn, 90 gələcək yazmalar üçün təxminən boş yer saxlayır. Sonradan həmin səhifənin daim 90 faiz dolu qalması demək deyil.

Aşağı fill factor bəzi split-ləri azalda bilər. Amma başlanğıcdan daha çox səhifə yaradır. Buna görə bütün indekslərə kor şəkildə aşağı dəyər vermək düzgün deyil.

# Nəyi ölçmədən qərar verməməliyik

SQL Server indeksin səhifə sayı, sıra uyğunsuzluğu və doluluğu barədə məlumat verə bilər:

T-SQL diaqnostika nümunəsi

```
SELECT
    page_count,
    avg_fragmentation_in_percent,
    avg_page_space_used_in_percent
FROM sys.dm_db_index_physical_stats
(
    DB_ID(), OBJECT_ID(N'dbo.Orders'),
    NULL, NULL, 'SAMPLED'
)
WHERE index_level = 0
  AND alloc_unit_type_desc = 'IN_ROW_DATA';
```

`page_count` səhifə sayıdır. `avg_fragmentation_in_percent` sıra uyğunsuzluğu göstəricisidir. `avg_page_space_used_in_percent` doluluq göstəricisidir. `index_level = 0` yarpaq səviyyəsini seçir.

`DB_ID()` cari bazanı, `OBJECT_ID` Orders obyektini müəyyən edir. `SAMPLED` nümunə götürməklə analiz aparır. `IN_ROW_DATA` sətir daxilində saxlanan məlumat hissəsidir. Sorğunun görünməsi üçün uyğun baza icazələri lazımdır.

Tək faizə baxma. Beş səhifəli indeksdə yüksək faiz praktik olaraq böyük problem olmaya bilər. Milyonlarla səhifəyə yayılmış boşluq isə daha əhəmiyyətli ola bilər.

**Qərarın əsası:** ölçü, doluluq və real sorğunun gördüyü iş birlikdə qiymətləndirilir. Diaqnostika özü də resurs sərf edir; hədəf cədvəldən başlamaq faydalıdır.

# Reorganize və rebuild eyni iş üsulu deyil

**Defraqmentasiya** quruluşu daha əlverişli formaya gətirmək işidir. SQL Server-də iki əsas indeks baxımı üsulu var:

| Üsul | Nə edir |
|---|---|
| REORGANIZE | Mövcud indeksin yarpaq səhifələrinin sırasını və sıxlığını mərhələli yaxşılaşdırır. |
| REBUILD | İndeksi yenidən qurur. Daha böyük resurs və əlavə sahə ehtiyacı yarada bilər. |

T-SQL alternativ əmrlər

```
ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REORGANIZE;

ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REBUILD;
```

Bunlar bir-birinin ardınca mütləq işlədiləcək siyahı deyil, iki alternativ nümunədir.

Reorganize indeks statistikasını ayrıca yeniləmir. Rebuild müvafiq indeks statistikasını yeniləyə bilər, amma cədvəlin bütün ayrı sütun statistikalarını avtomatik düzəltmir. Statistika dəyərlərin paylanması barədə plan seçiminə kömək edən məlumatdır.

**Online** baxım işləyərkən istifadəçi girişini daha çox davam etdirməyə imkan verir. Bütün kilidlərin yox olması demək deyil. Dəstək versiya, indeks növü və məhsul paketindən asılıdır. **Offline** baxım daha çox bloklanma yarada bilər.

**Nəticə:** resurs xərci ilə gözlənən sorğu qazancını müqayisə etmədən bütün indeksləri rebuild etmə.

# Hansı vəziyyətdə baxım fayda verə bilər

Sifariş hesabatı geniş tarix aralığını oxuyur və çoxlu səhifə gəzir. İndeksdə aşağı doluluq varsa daha sıx quruluş səhifə sayını azalda bilər. Burada baxımın real faydası ola bilər.

Başqa sorğu isə bir sifariş nömrəsini dərhal tapır. Yüksək fraqmentasiya faizi həmin dar axtarışın mütləq yavaş olduğunu göstərmir.

Köhnə şablonlarda 5 və 30 faiz kimi hədlər görünə bilər. Bunlar bütün sistemlər üçün dəyişməz qanun deyil. **Əvvəl yavaş sorğunu ölç, sonra problemin fiziki quruluşdan gəlib gəlmədiyini yoxla.**

Saxlama qurğusu da təsir edir. SSD elektron yaddaş əsasında, HDD isə mexaniki disk əsasında işləyir. Səhifələrin fiziki sırasının təsiri eyni olmaya bilər. Aşağı doluluğun daha çox səhifə və yaddaş xərci yaratması isə yenə qalır.

**Columnstore** sətirləri başqa qayda ilə, sütun əsaslı sıxılmış qruplarda saxlayan indeks ailəsidir. Orada silinmiş sətir payı və qrupların ölçüsü kimi başqa göstəricilər əhəmiyyətlidir. Rowstore fraqmentasiya faizini ora olduğu kimi tətbiq etmə.

### Özünü yoxla

Rebuild-dən sonra faiz yaxşılaşdı, sorğu vaxtı dəyişmədi. **Bu mümkündür.** Yavaşlığın əsas səbəbi başqa yerdə ola bilər. Məqsəd tək faizi deyil, real işi yaxşılaşdırmaqdır.

# PostgreSQL-də VACUUM və REINDEX fərqi

PostgreSQL bir sətir dəyişəndə köhnə versiyanı dərhal yox etməyə bilər. Çünki həmin versiyanı hələ oxuyan başqa iş ola bilər. Bir neçə sətir versiyası ilə paralel işi idarə etməyə **MVCC** deyilir.

Sadələşdirilmiş dəyişiklik

```
Evvel: Order 42, Total = 30.
Sonra: Order 42, Total = 35.

Kohne oxucu ucun 30 versiyasi hele lazim ola biler.
Yeni oxucu yeni qaydalara uygun 35-i gore biler.
```

Artıq heç bir aktiv işə lazım olmayan köhnə sətir versiyasına **dead tuple** deyilir. Tuple burada sətir versiyasıdır. Bu və başqa boşluqlar faydalı məlumatla müqayisədə saxlanmanı böyüdə bilər; artıq şişməyə **bloat** deyilir.

**VACUUM** təmizlənə bilən köhnə versiyaların yerini gələcək istifadə üçün azad edir. Adi VACUUM ümumən bütün faylı sıxıb boşluğu diskə qaytarmır. **ANALYZE** plan üçün statistikanı yeniləyir. **Autovacuum** bu işləri avtomatik aparan mexanizmdir.

**VACUUM FULL** cədvəli yenidən yazıb sıxlaşdıra bilər, amma güclü kilid və əlavə sahə tələb edir. **REINDEX** indeksin yeni quruluşunu yaradır. Bunlar bir-birinin adı dəyişmiş eyni əməliyyat deyil.

**Yadda saxla:** PostgreSQL versiya təmizliyi ilə SQL Server rowstore sıra fraqmentasiyasını eyniləşdirmə.

# 7 SOLID dəyişiklikləri nəzarətdə saxlamaq üçündür

Kafe tətbiqi böyüyür. Yeni endirim növü, başqa qəbz formatı, başqa baza və yeni bildiriş kanalı əlavə olunur. Hər kiçik dəyişiklik bütün proqramı pozmamalıdır.

**SOLID** bu problemi düşünmək üçün beş dizayn prinsipinin baş hərfləridir. Dizayn burada kodun hansı hissəsinin hansı işi gördüyünü və hissələrin bir-birinə necə bağlı olduğunu müəyyən etməkdir.

| Hərf | Əsas fikir |
|---|---|
| S | Bir-birindən ayrı dəyişmə səbəblərini qarışdırma. |
| O | Seçilmiş davranışı mövcud əsas axını dağıtmadan genişləndir. |
| L | Yeni növ köhnə müqavilənin vədini pozmasın. |
| I | İstifadəçini lazım olmayan metodlardan asılı etmə. |
| D | İş qaydasını dəyişən texniki detala möhkəm bağlama. |

Bu cədvəli əzbərləmək hələ prinsipləri bilmək deyil. Növbəti səhifələrdə hər prinsipin hansı dəyişiklik problemini həll etdiyini görəcəyik.

**Məqsəd:** daha çox sinif və interface yaratmaq yox, dəyişikliyin hara təsir edəcəyini aydınlaşdırmaqdır.

# S eyni sinfi niyə müxtəlif adamlar dəyişir

Bir InvoiceService qəbzin vergisini hesablayır, onu bazaya yazır, PDF görünüşünü qurur və email göndərir.

Sabah vergi qaydası dəyişir. Sonra dizayner qəbzin görünüşünü dəyişir. Sonra baza texnologiyası dəyişir. Hamısına görə eyni sinif dəyişirsə ayrı səbəblər bir yerə yığılıb.

**SRP**, Single Responsibility Principle, əlaqəli bir dəyişmə səbəbinə xidmət edən məsuliyyət sərhədi qurmağı istəyir.

Mümkün bölgü

```
InvoiceCalculator -> Meblegi ve vergini hesablayir.
InvoiceStore      -> Bazada saxlayir.
InvoiceRenderer   -> Qebzin gorunusunu hazirlayir.
InvoiceSender     -> Hazir qebzi gonderir.
```

İndi qəbzin rəngi dəyişəndə vergi hesabı dəyişməyə bilər. Baza dəyişəndə görünüş koduna toxunmaya bilərik.

Bu işləri ardıcıllaşdıran ayrıca xidmət də ola bilər: hesabla, saxla, göndər. Addımları əlaqələndirmək özü bir məsuliyyət ola bilər.

**SRP "hər sinifdə bir metod olsun" demək deyil.** Vergi hesablayan sinifdə bir neçə əlaqəli köməkçi metod normaldır. Əsas sual həmin metodların eyni səbəbə görə birlikdə dəyişib dəyişməməsidir.

Bir yerin içindəki hissələrin eyni məqsədə xidmət etməsinə **cohesion**, uyğunluq deyilir. SRP uyğun olmayan dəyişiklikləri ayırmaqla bunu yaxşılaşdırmağa kömək edir.

# O yeni endirim əsas sifariş axınını dəyişməsin

Normal müştəriyə 10, VIP müştəriyə 20 faiz endirim veririk. Sonra tələbə endirimi əlavə olunur. Hər yeni növ üçün Checkout-un içində uzun seçim məntiqi artırsa əsas axın da daim dəyişir.

Endirim hesabını ayrıca müqaviləyə çıxaraq:

C#

```
public interface IDiscount
{
    decimal Calculate(decimal total);
}

public class VipDiscount : IDiscount
{
    public decimal Calculate(decimal total)
        => total * 0.20m;
}
```

`decimal` dəqiq onluq ədəd tipidir. `0.20m` həmin tipdə 20 faiz dəyəridir. `Calculate` endirimin öz məbləğini qaytarır; yekun ödəniş `total - endirim` olur.

Checkout konstruktorla IDiscount alsın. Yeni StudentDiscount eyni müqaviləni yerinə yetirsə Checkout-un hesab axını dəyişmədən yeni variantı istifadə edə bilər.

Bu **OCP**, Open Closed Principle, üçün nümunədir: seçilmiş dəyişiklik istiqamətində genişlənməyə imkan veririk, sabit əsas məntiqə az toxunuruq.

Qeydiyyat və seçim yerinin dəyişməsi normaldır. OCP “heç bir fayl heç vaxt dəyişməsin” demir. İki sabit hal üçün sadə `switch` də kifayət edə bilər; hər seçimə dərhal ayrıca pattern qurmaq məcburi deyil.

# L yeni növ eyni vədi saxlamalıdır

Bir müqaviləmiz var: “etibarlı mətni verilən yerə yaz”. Proqram bu müqaviləyə güvənərək yazma metodunu çağırır.

İndi yalnız oxumağı bacaran obyekt həmin yazma müqaviləsini götürür və bütün çağırışlara “mən yaza bilmirəm” xətası verir. Adı və metod imzası uyğun olsa da davranış vədi pozulub.

**LSP**, Liskov Substitution Principle, yeni alt növün baza müqaviləsinin yerində istifadə ediləndə onu istifadə edən kodun düzgün gözləntilərini pozmamasını istəyir.

Müqavilə nəyi deyir

```
Qebul edilen giris: etibarli metn ve yazila bilen yol.
Ved edilen netice: metn yazilir ve ya ved edilen xeta gelir.

Yanlis yeni davranis:
    Etibarli girisde de "hec vaxt yaza bilmirem".
```

Çağırışdan əvvəl tələb olunan qaydaya **precondition**, uğurlu çağırışdan sonra vəd edilən nəticəyə **postcondition** deyilir. Obyektin etibarlı vəziyyətində qoruduğu qayda isə **invariant** adlanır.

Müqavilədə disk dolanda xəta ola biləcəyi deyilibsə, həmin xətanın baş verməsi təkbaşına LSP pozuntusu deyil. Əsas məsələ gözlənilməyən yeni məhdudiyyət və ya zəiflədilmiş vəddir.

**Praktik həll:** yalnız oxuyan obyektə yazma vədi yükləmə. Müqaviləni onun həqiqətən bacardığı əməliyyatlara uyğun seç. Növbəti prinsip bu bölgünü açır.

# I hər istifadəçiyə lazım olan müqaviləni ver

Kafedə bir hesabat xidməti yalnız sifariş oxuyur. Amma ona verilən IOrderStore müqaviləsi oxuma, əlavə etmə, silmə, arxivləmə və başqa işlərlə doludur.

Hesabat xidməti istifadə etmədiyi yazma əməliyyatlarına da bağlı olur. Yalnız oxuyan başqa implementasiya isə yalandan yazma metodları təqdim etməyə məcbur qala bilər.

C# müqavilə nümunələri

```
public interface IOrderReader
{
    string Read(int id);
}

public interface IOrderWriter
{
    void Save(string order);
}
```

Hesabat IOrderReader alır. Yazma xidməti IOrderWriter alır. Ehtiyac varsa bir konkret sinif hər iki müqaviləni həyata keçirə bilər. `void` metodun nəticə dəyəri qaytarmadığını bildirir. Bu kiçik nümunədə sifariş mətnlə təmsil edilir.

Bu **ISP**, Interface Segregation Principle, üçün nümunədir. Burada “client”, müştəri, interface-dən istifadə edən kod deməkdir; kafedəki insan deyil.

ISP “hər interface-də yalnız bir metod olsun” demir. Birlikdə istifadə olunan oxuma metodlarını eyni müqavilədə saxlamaq normal ola bilər.

**Nəticə:** interface-i onu istifadə edən kodun real ehtiyacına görə böl. Sadəcə xırdalamaq üçün bölmək də əlavə qarışıqlıq yarada bilər.

# D iş qaydası konkret baza sinfinə bağlanmasın

Sifarişi tamamlamaq biznes işidir. SQL bağlantısını açmaq isə həmin işin texniki detalıdır. Sifarişin əsas məntiqi birbaşa konkret SQL kitabxanasının təfərrüatına bağlanarsa texnologiya dəyişəndə əsas məntiq də dəyişə bilər.

İşin ehtiyacını bir müqavilə ilə göstəririk:

Asılılıq istiqaməti

```
Sifaris isi -> IOrderStore <- SqlOrderStore

Sifaris isi saxlamani teleb edir.
SqlOrderStore hemin telebi SQL ile yerine yetirir.
```

Bu **DIP**, Dependency Inversion Principle, üçün nümunədir. İş qaydası çox vaxt **high-level**, onu yerinə yetirən texniki detal **low-level** adlandırılır. Hər ikisini uyğun abstraksiya ilə bağlayırıq. Abstraksiya burada “sifarişi saxla” ehtiyacını ifadə edən müqavilədir.

**DIP ilə DI fərqi:** DIP dizaynda nəyə bağlı olduğumuzu seçir. DI isə seçilmiş asılılığı obyektə kənardan vermək üsuludur. Interface yaratmadan da constructor-la konkret SQL sinfi vermək mümkündür; bu DI-dır, amma texniki detaldan asılılığı aradan qaldırmır.

Qatların “controller service-i çağırır” formasında düzülməsi təkbaşına DIP zəmanəti deyil. Müqavilənin özü də konkret SQL texnologiyasının tipləri ilə doludursa detal yenə əsas işə sızmış ola bilər.

# Beş prinsipi eyni dəyişiklik üzərində yoxla

Yeni qəbz formatı əlavə edirik. Özümüzdən bunları soruşaq:

**S:** görünüş dəyişdiyi üçün vergi və saxlanma siniflərini də dəyişirəmmi? Edirəmsə məsuliyyət sərhədi qarışmış ola bilər.

**O:** yeni qəbz implementasiyası əlavə edib mövcud sifariş axınını saxlayırammı?

**L:** yeni implementasiya eyni qəbul edilən girişlər üçün eyni vədi yerinə yetirirmi? Yoxsa köhnə kod indi əlavə xüsusi hallar yazmağa məcburdur?

**I:** qəbzi yalnız göstərən kodu çap və silmə kimi lazım olmayan əməliyyatlardan asılı etmişəmmi?

**D:** sifariş işi qəbz müqaviləsini bilir, yoxsa konkret PDF kitabxanasının iç quruluşunu?

### Bir dəyişikliklə məşq

Əvvəl TextReceipt verən sifariş axını qur. Sonra PdfReceipt əlavə et. Sifariş kodunda hansı dəyişikliklərin lazım olduğunu yaz. Məqsəd interface sayını artırmaq yox, yeni variantın təsir etdiyi yerləri azaltmaqdır.

**Yadda saxla:** SOLID düzgün proqramı avtomatik yaradan düstur deyil. Dəyişiklik səbəbi, müqavilə və asılılıq istiqaməti barədə düşünmək üçün vasitədir. Əlavə abstraksiya problemi asanlaşdırmırsa onun xərci də nəzərə alınmalıdır.

# 8 Pattern təkrarlanan problem üçün həll quruluşudur

Kafe tətbiqində qəbzin mətn və PDF variantı, çatdırılmanın adi və sürətli variantı, ödənişin müxtəlif üsulları yarandı. Hər yerdə eyni dəyişiklik problemlərini təkrar görürük.

**Design pattern** belə təkrarlanan problemə verilən ümumi həll quruluşudur. Hazır köçürüləcək kod deyil. Onun adını bilməzdən əvvəl həll etdiyi problemi başa düşmək lazımdır.

Dəyişən hissəni əvvəl seç

```
Hansi davranis deyisir?       -> Strategy dusun.
Hansi obyekt alinmalidir?     -> Factory dusun.
Movcud interfeys uygun deyil? -> Adapter dusun.
Eyni ise elave davranis?      -> Decorator dusun.
```

Bu adlar seçimə kömək edir. Eyni quruluşu hər problemə tətbiq etmək lazım deyil.

Klassik **GoF** kataloqunda 23 pattern var. GoF həmin kitabın dörd müəllifinə verilən addır. Pattern-lər üç ailəyə ayrılır: **creational** yaradılmanı, **structural** quruluşu, **behavioral** davranış və ünsiyyəti təşkil edir.

Bu dərsdə əvvəl backend-də asan rast gəlinən problemləri açacağıq. Sonra qalan pattern-ləri konkret nümunə ilə bir-birinə bağlayacağıq.

# Strategy dəyişən hesabı ayrıca saxlayır

Adi müştəri və VIP üçün fərqli endirim hesablayırıq. Məqsəd hər ikisində eynidir: endirim məbləğini tapmaq. Dəyişən hissə hesablama qaydasıdır.

SOLID dərsindəki `IDiscount` nümunəsini xatırla. Hər variant `Calculate(total)` təqdim edir. Əsas sifariş kodu seçilmiş variantı çağırır:

C# istifadə fraqmenti

```
decimal discountAmount = discount.Calculate(total);
decimal payable = total - discountAmount;
```

`discount` constructor-la verilmiş IDiscount obyektidir. `discountAmount` endirim məbləği, `payable` ödəniləcək məbləğdir.

Bu quruluş **Strategy** pattern-dir. Əsas kod eyni məqsədli alqoritmlərdən birini istifadə edir. **Alqoritm** nəticəyə çatmaq üçün addımlar qaydasıdır.

Əlaqə

```
Checkout -> IDiscount
                |
                +-- NormalDiscount
                +-- VipDiscount
                +-- StudentDiscount
```

Yeni endirim gələndə onun hesabı ayrıca yazılır. Checkout-un əsas hesablaması dəyişməyə bilər. Hansı strategiyanın seçilməsi isə ayrıca seçim yerinin işidir.

**Nə vaxt uyğundur:** eyni işi görən qaydaların variantları artır və müstəqil dəyişir. Cəmi iki sabit, sadə hal varsa bir şərt ifadəsi də kifayət edə bilər. Pattern-in əlavə quruluş xərci də var.

# Factory hansı obyektin alınacağını bir yerə toplayır

Müştəri qəbzi PDF istəyir, kassir isə mətn qəbzi istəyir. Hər istifadə yerində “PDF-dirsə bunu yarat, mətndirsə onu yarat” kodunu təkrarlamaq istəmirik.

**Factory** obyekt yaratma və ya uyğun obyekti vermə məntiqini ayrıca yerə toplayır. Sadə seçim metoduna çox vaxt **Simple Factory** deyilir.

Sadə factory fikri

```
CreateReceipt("text") -> TextReceipt obyekti.
CreateReceipt("pdf")  -> PdfReceipt obyekti.
```

Klassik **Factory Method** bir qədər fərqlidir: baza sinif ümumi işi saxlayır, hansı məhsul obyektinin yaradılacağını alt sinfin əvəz edə bildiyi metoda verir.

Factory Method əlaqəsi

```
ReportJob: "Exporter yarat, sonra export et."
    |
    +-- CsvJob CreateExporter metodunu deyisir.
    +-- PdfJob CreateExporter metodunu deyisir.
```

**Baza sinif** ortaq davranışı təqdim edən sinifdir. **Alt sinif** ondan törəyib müəyyən addımı dəyişən sinifdir. Export məlumatı başqa formata çıxarmaqdır. CSV sadə cədvəl mətn formatıdır.

Hər adı Create olan metod GoF Factory Method deyil. Əsas məntiq yaradılma qərarının alt sinfə verilən dəyişmə nöqtəsi olmasıdır.

**Strategy ilə fərq:** Strategy davranışın necə hesablanmasına, Factory isə hansı obyektin yaradılıb verilməsinə aiddir. Birlikdə işləyə bilərlər.

# Abstract Factory uyğun məhsul ailəsi yaradır

Kafe proqramının açıq və tünd görünüşü var. Hər görünüşün düyməsi və seçim qutusu bir-birinə uyğun olmalıdır.

Bir factory yalnız bir obyekt yox, uyğun bir neçə obyekt yarada bilər:

Məhsul ailəsi

```
LightFactory
    CreateButton()   -> aciq duymə.
    CreateCheckbox() -> aciq secim qutusu.

DarkFactory
    CreateButton()   -> tund duymə.
    CreateCheckbox() -> tund secim qutusu.
```

Bu **Abstract Factory** pattern-dir. Məqsəd uyğun məhsullar ailəsini birlikdə yaratmaqdır. Onu sadəcə “factory-lərin idarəçisi” kimi əzbərləmək əsas məntiqi gizlədir.

Yeni görünüş ailəsi əlavə etmək rahatlaşır. Amma ailəyə yeni məhsul növü, məsələn slider, əlavə etsən bütün factory-lərin müqaviləsi dəyişə bilər. Slider dəyəri sürüşdürərək seçən görünüş elementidir.

**DI ilə əlaqə:** container obyektlərin əlaqələrini və lifetime-ını qura bilər. Factory isə konkret girişə görə uyğun variantın alınmasını aydın ifadə edə bilər. .NET DI da seçim funksiyası və açarlı xidmətlər kimi üsullar təqdim edir. “DI işləmə vaxtında seçim edə bilmir” düzgün iddia deyil.

Əsas sual ad deyil: bir obyekt seçirsən, alt sinfə yaradılma addımı verirsən, yoxsa uyğun məhsul ailəsi yaradırsan? Bu fərq factory növünü başa salır.

# Adapter uyğun gəlməyən çağırışı çevirir

Köhnə printer kitabxanasında metod `SendText` adlanır. Yeni sifariş kodumuz isə `IPrinter.Print` müqaviləsi gözləyir. İş eynidir, giriş formaları uyğun deyil.

**Adapter** bu uyğunlaşdırmanı ayrıca obyektə yığır:

C#

```
public class PrinterAdapter : IPrinter
{
    private LegacyPrinter oldPrinter;

    public PrinterAdapter(LegacyPrinter oldPrinter)
    {
        this.oldPrinter = oldPrinter;
    }

    public void Print(string text)
    {
        oldPrinter.SendText(text);
    }
}
```

Bu fraqmentdə IPrinter-in `void Print(string text)` müqaviləsini, LegacyPrinter-in isə `SendText(string text)` metodunu təqdim etdiyini qəbul edirik. Legacy köhnədən qalmış sistem deməkdir.

Sifariş kodu IPrinter-i tanıyır. Adapter çağırışı köhnə metodun formasına çevirir. Köhnə kitabxananı bütün tətbiqə yaymırıq.

Uyğunlaşdırma yalnız metodun adını dəyişmək olmaya bilər. Məsələn, yeni müqavilə manat, köhnə sistem qəpik qəbul edirsə məbləğ və yuvarlaqlaşdırma da düzgün çevrilməlidir.

**Yadda saxla:** Adapter-in əsas niyyəti fərqli müqavilələri uyğunlaşdırmaqdır.

# Decorator eyni müqaviləyə əlavə davranış verir

Qiymət xidmətimiz işləyir. İndi hər qiymət istəniləndə bir qeyd yazmaq istəyirik. Əsas qiymət hesabını dəyişmədən onu başqa obyektlə sarıya bilərik.

C#

```
public class LoggedPrice : IPrice
{
    private IPrice inner;

    public LoggedPrice(IPrice inner)
    {
        this.inner = inner;
    }

    public decimal Get(int id)
    {
        Console.WriteLine("Qiymet istendi");
        return inner.Get(id);
    }
}
```

IPrice müqaviləsində `decimal Get(int id)` metodu olduğunu qəbul edirik. `inner` əsl işi görən qiymət xidmətidir. Yeni obyekt eyni müqaviləni saxlayır, çağırışa qeyd əlavə edir və işi inner-ə ötürür. Bu ötürməyə **delegation** deyilir.

Bu **Decorator** pattern-dir. Eyni yanaşma ölçmə və uyğun hallarda keş üçün işlədilə bilər. **Keş** əvvəl alınmış nəticəni təkrar istifadə üçün saxlamaqdır; köhnəlmiş nəticənin nə vaxt silinəcəyi ayrıca seçilir.

**Adapter ilə fərq:** Adapter müqaviləni uyğunlaşdırır. Decorator eyni müqavilə ətrafında əlavə davranış qurur.

# Mediator əlaqələndirməni vasitəçiyə verir

Sifariş yaradılır. Ehtiyat yoxlanmalı, ödəniş təşkil olunmalı, qəbz hazırlanmalıdır. Hər obyekt bütün digərlərini birbaşa tanıyırsa əlaqə şəbəkəsi çətinləşə bilər.

**Mediator** qarşılıqlı işi əlaqələndirən vasitəçidir. Komponentlər bir-birinin bütün təfərrüatını bilmədən onun üzərindən işləyə bilər.

Sadələşdirilmiş əlaqə

```
Sifaris isteyi
    |
    v
Vasiteci
    +-- uygun isleyiciye yoneldir.
    +-- lazim olan elaqelendirməni aparir.
```

Konkret işi emal edən hissəyə **handler**, işləyici deyilir. .NET-də MediatR adlı kitabxana var, amma pattern-in özü həmin kitabxanadan asılı deyil.

Mühüm sərhəd var: A vasitəçi ilə B-ni, B isə vasitəçi ilə yenə A-nı çağırırsa çağırış dövrəsi yenə qala bilər. Sadəcə araya vasitəçi qoymaq səhv məsuliyyət bölgüsünü avtomatik düzəltmir.

Vasitəçi bütün biznes işlərini özünə yığıb həddindən artıq böyük obyektə də çevrilməməlidir.

**Nəticə:** Mediator əlaqələri idarə etməyi mərkəzləşdirə bilər. Hər dairəvi asılılığın və hər mürəkkəb sistemin avtomatik həlli deyil.

# State Strategy və Template Method fərqi

### State cari vəziyyətə görə davranışı dəyişir

Sifariş əvvəl Pending, sonra Paid, sonra Shipped olur. Bunlar gözləmədə, ödənilib və göndərilib vəziyyətləridir. Göndərilmiş sifarişdə bəzi əməliyyatlara artıq icazə verilmir. **State** hər vəziyyətin davranışını ayrıca təşkil edir. Bir vəziyyətdən başqasına keçidə **transition** deyilir.

Strategy-də əsas sual “hansı hesablama üsulunu seçək?” idi. State-də “obyekt hazırda hansı vəziyyətdədir və nə edə bilər?” sualıdır.

### Template Method ardıcıllığı sabit saxlayır

Fayl idxalı həmişə oxu, yoxla, çevir, saxla addımlarından keçir. CSV və JSON üçün oxuma addımı fərqlənə bilər. **Template Method** ümumi ardıcıllığı baza sinifdə saxlayır, müəyyən addımları alt sinfə verir. JSON məlumatın açar və dəyərlərlə yazıldığı mətn formatıdır.

### Command əməliyyat istəyini obyekt edir

“Sifarişi ləğv et” istəyini əməliyyat növü və sifariş nömrəsi olan obyekt kimi saxlaya bilərik. **Command** həmin istəyi sonradan işlətməyə, növbəyə qoymağa və qeyd etməyə uyğun edir. **Növbə** işlərin sonradan işlənmək üçün saxlandığı ardıcıllıqdır.

Command yaratmaq hər işi geri almağı avtomatik mümkün etmir. Göndərilmiş email üçün sadə undo yoxdur.

# Digər struktur patternlərini nümunə ilə ayıraq

### Bridge iki ayrı dəyişən istiqaməti ayırır

Bildirişin növü sadə və təcili ola bilər. Göndərmə kanalı email və SMS ola bilər. Hər kombinasiyaya ayrıca sinif yaratmaq əvəzinə bildiriş məntiqi ilə kanalı ayrı saxlayırıq. Bu **Bridge** ideyasıdır. İki tərəf ayrıca genişlənə bilər.

### Composite tək elementlə ağacı eyni qayda ilə işlədir

Bir menyu bölməsində məhsullar və başqa alt bölmələr ola bilər. Həm tək məhsula, həm bütöv bölməyə eyni “göstər” əməliyyatı tətbiq etmək istəyirik. **Composite** element və element qrupuna ortaq müqavilə verir. İç-içə qruplar ağac quruluşudur.

### Facade mürəkkəb alt sistemi sadə girişlə təqdim edir

Sifariş tamamlamaq üçün bir neçə texniki hissə çağırılmalıdır. **Facade** istifadəçiyə sadə giriş verib həmin təfərrüatları arxada əlaqələndirir. “CompleteOrder” çağırışı bunun nümunəsi ola bilər; transaction zəmanəti ayrıca qurulur.

### Flyweight dəyişməz ortaq məlumatı paylaşır

Minlərlə simvol eyni şrift məlumatını ayrıca saxlamaq əvəzinə paylaşa bilər. **Flyweight** belə ortaq hissəni ayırır. Hər simvolun ekrandakı yeri isə ayrıca verilir. Paylaşılan məlumatı nəzarətsiz dəyişmək düzgün deyil.

### Proxy obyektə çıxışı idarə edir

**Proxy** əsl obyektin qarşısında dayanıb giriş icazəsini, uzaq çağırışı və ya gec yaradılmanı idarə edə bilər. Quruluşu Decorator-a bənzəyə bilər. Burada əsas niyyət obyektə çatmağı idarə etməkdir.

# Yaradılma və vəziyyət saxlama üçün digər vasitələr

### Builder mürəkkəb nəticəni addım-addım qurur

Hesabatın başlığı, bölmələri və filtrləri verilir, sonda Build çağırılır. **Builder** qurulmanı mərhələlərə ayırır və tamamlananda məcburi hissələri yoxlaya bilər. Metodları zəncirlə yazmaq təkbaşına Builder olması demək deyil; əsas niyyət etibarlı nəticə qurmaqdır.

### Prototype hazır nümunədən kopya yaradır

Mürəkkəb şablonu sıfırdan qurmaq əvəzinə kopyalaya bilərik. **Prototype** buna xidmət edir. Daxili siyahıları da eyni istinadla paylaşan kopya **shallow copy**-dir. Onları ayrıca kopyalamaq **deep copy** yanaşmasıdır. Hansı hissənin paylaşılacağı düşünülməlidir.

### Singleton seçilmiş mühitdə bir nüsxəni paylaşır

Klassik **Singleton** bir nüsxə və ona çıxışı təşkil edir; çox vaxt proqramın hər yerindən əlçatan qlobal girişlə yazılır. DI singleton isə bir container-də paylaşılma qaydasıdır. Bunları eyniləşdirmə. İki prosesin ayrıca nüsxəsi olur. Nüsxənin təhlükəsiz yaradılması onun bütün metodlarının paralel istifadəyə təhlükəsiz olduğunu sübut etmir.

### Iterator daxili saxlanmanı açmadan gəzməyə imkan verir

`foreach` siyahının elementlərini gəzir. **Iterator** istifadəçiyə elementləri ardıcıl əldə etməyə imkan verir; onların daxildə massiv, ağac və ya başqa quruluşda olması gizli qala bilər.

### Memento əvvəlki vəziyyəti saxlayır

Redaktorda son dəyişikliyi geri almaq üçün əvvəlki vəziyyətin surətini saxlayırıq. **Memento** bunu obyektin daxili quruluşunu hər yerə açmadan təşkil edir. Böyük surətlərin yaddaş xərci vardır.

# Hadisə zəncir və qayda emalı

### Observer dəyişiklik barədə abunə olanlara xəbər verir

Qiymət dəyişəndə ekran və başqa izləyicilər yenilənsin. **Observer** hadisəni yayan obyektlə həmin hadisəyə abunə olanları əlaqələndirir. C# event mexanizmi bu ideyaya nümunədir. Uzunömürlü yayıcı artıq lazım olmayan abunəçini saxlaya bilər; abunəliyin bitirilməsi nəzərə alınır.

### Chain of Responsibility sorğunu işləyicilərdən keçirir

Sorğu A, B və C mərhələlərindən keçir. B işi tamamlayıb dayandıra və ya C-yə ötürə bilər. **Chain of Responsibility** bu quruluşdur. Middleware dərsində gördüyümüz axın buna yaxın nümunədir. Sıra dəyişəndə nəticə də dəyişə bilər.

### Interpreter kiçik bir dilin qaydalarını işlədir

`total > 100 AND vip` kimi sadə qaydanın mənasını müəyyən edib qiymətləndiririk. **Interpreter** həmin dilin hissələrini təmsil edir və şərh edir. **Grammar** hansı ifadələrin düzgün qurulduğunu bildirən qaydalardır. Çox böyük dil üçün bu quruluş ağırlaşa bilər.

### Visitor sabit obyekt quruluşuna yeni əməliyyat verir

Məlumat ağacındakı tiplər sabitdir, amma onları çap etmək, hesablamaq və yoxlamaq kimi əməliyyatlar artır. **Visitor** əməliyyatı həmin tiplərdən ayrı təşkil edir. Yeni əməliyyat əlavə etmək rahatlaşa, yeni obyekt tipi əlavə etmək isə bütün visitor-lərə təsir edə bilər.

**Nəticə:** bu pattern-lərin hamısı “kod səliqəli olsun” deyə eyni işi görmür. Hərəsi konkret dəyişmə problemini başqa yerdə saxlayır.

# Backend-də eşidəcəyin əlavə patternlər

Bunlar klassik 23 GoF pattern ilə eyni siyahı deyil, amma müsahibədə tez-tez yanaşı çəkilir.

**Repository** domain obyektlərinə çıxışı bir sərhəddə toplayır. Domain biznes anlayışları və qaydalarıdır. “Müştərinin aktiv sifarişlərini ver” əməliyyatı saxlanma təfərrüatından ayrılmış müqavilə ola bilər.

**Unit of Work** bir iş ərzində dəyişən obyektləri izləyib yazmaları əlaqələndirir. EF Core DbContext bu imkanların çoxunu artıq verir. Üzərinə əlavə ümumi repository yazmaq hər layihədə məcburi deyil.

**CQRS** dəyişiklik əmrləri ilə oxuma sorğularının məsuliyyətini və modelini ayırır. Ayrı baza və ayrı mikroservis məcburi deyil. Əsas fikir yazma modeli ilə oxuma ehtiyacının həmişə eyni formaya sıxışdırılmamasıdır.

**Outbox** sifariş dəyişikliyi ilə göndəriləcək hadisəni eyni baza transaction-unda saxlayır. Ayrı işçi sonradan hadisəni göndərir. Bu, “sifariş yazıldı, xəbər itdi” problemini həll etməyə kömək edir. Amma hadisə təkrar göndərilə bilər.

**Idempotency** eyni iş təkrar gələndə əlavə yanlış təsir yaratmamaqdır. Məsələn, eyni hadisə nömrəsi ilə iki dəfə ödəniş yazmamaq. “SaveChanges et, sonra Publish et” təkbaşına iki ayrı sistem arasında birlikdəlik zəmanəti yaratmır. Publish hadisəni yaymaq deməkdir.

# Adı deyil problemi tanımaq üçün məşq

| Problem | Düşünüləcək pattern |
|---|---|
| Yeni endirim hesabı əlavə olunur | Strategy |
| Uyğun qəbz obyektini bir yerdən almaq lazımdır | Factory yanaşması |
| Köhnə printerin metodu yeni müqaviləyə uyğun deyil | Adapter |
| Əsas qiymət koduna toxunmadan qeyd əlavə edilir | Decorator |
| Obyektin vəziyyəti hansı əməliyyatın mümkün olduğunu dəyişir | State |
| Mürəkkəb alt sistemə sadə giriş lazımdır | Facade |

### Özünü yoxla

“Adapter ilə Decorator eyni deyil” cümləsinin səbəbini de. **Adapter fərqli müqaviləni uyğunlaşdırır; Decorator eyni müqavilə ətrafına davranış əlavə edir.**

“Strategy ilə Factory fərqi nədir?” sualına cavab ver. **Biri dəyişən davranışın icrasını ayırır, o biri uyğun obyektin alınmasını təşkil edir.**

Yeni pattern istifadə edəndə onun çıxarılması ilə hansı konkret problemin geri qayıdacağını izah edə bil. Cavab yalnız “kod çirkin olar” olmamalıdır.

**Əsas nəticə:** pattern seçimi problemin yeri və dəyişən hissəsi ilə əsaslandırılır. Adı əzbərləmək həmin qərarı əvəz etmir.

# 9 Async gözləyərkən threadi boş saxlamağa kömək edir

Kafe tətbiqi uzaq ödəniş xidmətindən cavab gözləyir. Cavab gələnə qədər proqramın həmin yerdə hesablayacağı iş yoxdur. Sadəcə başqa sistemin işini bitirməsi lazımdır.

İki yanaşma ola bilər. Kod icra edən axını gözləmədə tuturuq. Yaxud cavab gələndə davam etmək üçün məlumatı saxlayıb axını başqa işə buraxırıq.

**Thread** prosesdə kod icra edən axındır. **Asinxron gözləmə** müəyyən əməliyyatın tamamlanmasını gözləyərkən həmin thread-i bloklamamağa imkan verir.

Gözləmə fərqi

```
Bloklayan gozleme:
    Thread bu yerde gozleyir, basqa is gormur.

Asinxron gozleme:
    Davamin haradan olacagi saxlanir.
    Cavab hazir olanda davam kodu isleyir.
```

Bu, uzaq xidmətin öz işini avtomatik sürətləndirmir. Cavab yenə vaxtında gəlir. Qazanc odur ki, gözləmə zamanı thread başqa iş üçün əlçatan qala bilər.

**Bir əməliyyatın cavab müddəti ilə serverin eyni müddətdə neçə işə xidmət etməsi ayrı göstəricilərdir.** Birincisinə latency, ikincisinə throughput deyilir.

# Task işi təmsil edir amma threadin özü deyil

Ödəniş başladılır. Dərhal yekun cavab yoxdur, amma onun tamamlanmasını təmsil edən obyekt alırıq. .NET-də belə tamamlanma üçün **Task** istifadə olunur.

`Task` nəticə dəyəri olmayan tamamlanmanı, `Task<int>` isə tamamlananda tam ədəd verən işi təmsil edə bilər. `<int>` nəticənin tipini göstərir.

Task barədə soruşduğumuz məlumat

```
Is bitibmi?
Ugurlu bitibse netice nedir?
Xeta ile bitibmi?
Legv olunubmu?
```

Task-ın olması bütün gözləmə müddətində ayrıca thread tutulması demək deyil. Şəbəkə cavabını əməliyyat sistemi izləyib hazır olanda xəbər verə bilər.

**ThreadPool** .NET-in qısa işlər üçün idarə etdiyi işçi thread-lər mexanizmidir. `Task.Run` işi adətən həmin pool-da planlayır. Amma bütün Task-lar Task.Run işi deyil. Artıq tamamlanmış Task və xarici cavabı gözləyən Task da mümkündür.

**Yadda saxla:** Task “bu əməliyyatın tamamlanmasını və nəticəsini izləyə bilərəm” deməkdir. “Bu əməliyyatın həmişə ayrıca bir thread-i var” demək deyil.

# Async və await işləyəndə kod hansı sırada gedir

Bu kiçik C# nümunəsini addım-addım oxuyaq:

C#

```
static async Task DemoAsync()
{
    Console.WriteLine("A");
    await Task.Delay(1000);
    Console.WriteLine("B");
}

Task work = DemoAsync();
Console.WriteLine("C");
await work;
```

Əvvəl DemoAsync çağırılır və **A** yazılır. `Task.Delay(1000)` təxminən 1000 millisaniyə sonra tamamlanan gözləmə yaradır. Bir saniyə 1000 millisaniyədir.

Delay hələ bitmədiyi üçün `await` yerində metodun davamı saxlanır və çağıran kod davam edə bilir. Ona görə **C** yazılır. Delay tamamlananda metod qaldığı yerdən davam edir və **B** yazılır. Gözlənən sıra **A, C, B**-dir.

`async` metodda belə gözləmələr yazmağa imkan verir. Metod çağırılan kimi ilk tamamlanmamış await-ə qədər adi qaydada işləyir. **Async yazmaq kodu əvvəldən avtomatik başqa thread-ə keçirmir.**

Əgər await olunan Task artıq bitibsə metod dayanmadan davam edə bilər. Hər await mütləq başqa thread-ə keçid demək deyil.

# Davam harada saxlanır və I O ilə CPU niyə fərqlidir

Metod gözləyəndə hara qayıdacağını və lazım olan dəyərləri saxlamalıdır. Kompilyator, C# mətnini işləyən formaya çevirən alət, bunun üçün **state machine** yaradır. Bu, metodun hansı mərhələdə olduğunu saxlayan mexanizmdir. Gözləmədən sonrakı kod **continuation**, davam kodu adlanır.

İndi işin əsas vaxtının hara getdiyini ayıraq:

| İşin növü | Əsas vaxt nəyə gedir |
| --- | --- |
| I/O-bound | Şəbəkə, disk və ya baza cavabını gözləməyə. |
| CPU-bound | Prosessorda hesablama aparmağa. |

**I/O** məlumatın proqramdan xaricə çıxması və ya xaricdən gəlməsidir. **CPU** hesablamaları yerinə yetirən prosessordur.

Şəkli serverdən almaq I/O üstünlüklü, onun üzərində ağır hesablama etmək CPU üstünlüklü ola bilər.

C# istifadə fraqmentləri

```
// Hazir asinxron I/O emeliyyatini gozle:
string text = await http.GetStringAsync(url);

// Lazim olanda CPU isini pool-da planla:
int value = await Task.Run(() => Calculate());
```

`http` .NET-dən HTTP sorğuları göndərən HttpClient obyektidir. `url` ünvan, `Calculate` hesablayan metodudur. Sinxron baza çağırışını Task.Run ilə sarımaq onu həqiqi asinxron baza metoduna çevirmir. Serverdə əlavə Task.Run ümumi hesablama gücünü artırmır.

# WhenAll müstəqil gözləmələri üst üstə salır

Qiymət sorğusu 300, ehtiyat sorğusu 500 millisaniyə gözləyir. Biri o birinin nəticəsindən asılı deyil.

C# ardıcıl variant

```
var price = await GetPriceAsync();
var stock = await GetStockAsync();
```

İkinci iş birinci bitəndən sonra başlayır. Sadə modeldə gözləmə `300 + 500 = 800` millisaniyədir.

İndi əvvəl hər ikisini başladaq:

C# müstəqil işlər üçün variant

```
var priceTask = GetPriceAsync();
var stockTask = GetStockAsync();

await Task.WhenAll(priceTask, stockTask);

var price = await priceTask;
var stock = await stockTask;
```

`GetPriceAsync` qiyməti, `GetStockAsync` ehtiyatı əldə edən nümunə metod adlarıdır. Onların əvvəl çağırılması işləri başladır. **WhenAll** verilmiş bütün Task-lar tamamlananda tamamlanan Task verir. Onları avtomatik ayrıca CPU nüvələrində işlətmək vədi vermir.

İdeal sadə gözləmə modelində ümumi müddət təxminən ən uzun gözləməyə, 500 millisaniyəyə yaxın ola bilər. Həqiqi vaxt xarici xidmətlərdən və limitlərdən asılıdır.

**Şərt:** işlər müstəqil olmalıdır. Eyni DbContext-də iki paralel baza əməliyyatını bu cür işlətmək olmaz. Eyni context eyni anda bir neçə əməliyyata uyğun deyil.

# Concurrency ilə parallelism fərqi

**Concurrency** bir neçə işin vaxt baxımından üst-üstə irəliləməsidir. Qiymət cavabını gözləyərkən ehtiyat sorğusunu başlatmaq bunun nümunəsidir. Hər an bir thread-də kod işləməsi də mümkün ola bilər.

**Parallelism** iş hissələrinin həqiqətən eyni anda müxtəlif icra resurslarında hesablanmasıdır. Bir neçə prosessor nüvəsində müxtəlif şəkillərin emalı nümunədir.

İki müxtəlif fikir

```
Concurrency:
    Iki xarici cavabi eyni vaxt araliginda gozleyirik.

Parallelism:
    Iki hesablamani heqiqeten eyni anda icra edirik.
```

Async əsasən gözləmə ilə işləmə üsuludur. Paralellik isə eyni anda hesablama aparmağa aiddir. Onlar birlikdə istifadə edilə bilər, amma bir-birinin adı deyil.

Hər işi limitsiz eyni anda başlatmaq da düzgün deyil. 50 min sorğunu birdən xarici xidmətə göndərmək həmin xidməti və bağlantı ehtiyatını tükədə bilər.

Buna görə bəzi işlərdə maksimum paralel əməliyyat sayı seçilir. Məsələn, eyni anda yalnız 4 fayl endirilsin. Dörd rəqəmi universal düzgün rəqəm deyil; xidmətin limiti və resurslar üzrə seçilir.

**Yadda saxla:** “bir neçə iş başladı” deyəndə onların gözləmələri üst-üstə düşür, yoxsa hesablamaları həqiqətən paralel işləyir, bunu dəqiqləşdir.

# Xəta olanda WhenAll və WhenAny nə edir

Bir Task xəta ilə bitə bilər. Xəta obyektinə **exception** deyilir. Onu await edəndə xəta çağıran koda ötürülə bilər.

C#

```
try
{
    await Task.WhenAll(priceTask, stockTask);
}
catch (Exception error)
{
    Console.WriteLine(error.Message);
}
```

`try` yoxlanılan iş blokudur. `catch` uyğun xətanı tutur. `error.Message` xətanın mesajıdır.

WhenAll işlərin hamısının tamamlanmasını gözləyir. Biri xəta verəndə qalanları avtomatik dayandırmır. Ümumi Task bütün xətaları öz Exception məlumatında saxlaya bilər; await isə adətən həmin xətalardan birini yenidən atır. Hamısı lazımdırsa ümumi Task-ın xəta toplusuna baxılır.

**WhenAny** ilk tamamlanan Task-ı verir. Tamamlanma uğur, xəta və ya ləğv ola bilər. Verilən Task-ı ayrıca await etmək onun həqiqi nəticəsini və ya xətasını almaq üçündür.

WhenAny də qalan işləri avtomatik ləğv etmir. “Birincisi hazırdır” ilə “o biriləri dayandırıldı” ayrı hadisələrdir.

**Əsas nəticə:** tamamlanmanı izləmək, xətanı almaq və digər işləri ləğv etmək ayrı qərarlardır.

# CancellationToken işi zorla öldürmür

Müştəri gözləməkdən vaz keçir. Və ya iş üçün ayrılmış 5 saniyə bitir. Davam etmək istəmədiyimizi əməliyyata bildirmək lazımdır.

**CancellationToken** ləğv istəyi barədə siqnalı daşıyır. **CancellationTokenSource** həmin siqnalı yaradan və ləğv edən obyektdir.

C#

```
using var stop = new CancellationTokenSource();
stop.CancelAfter(TimeSpan.FromSeconds(5));

string text = await http.GetStringAsync(url, stop.Token);
```

`CancelAfter` müddət bitəndə ləğv istəyir. `TimeSpan` vaxt müddətini bildirir. Token http metoduna ötürülür ki, həmin əməliyyat siqnalı nəzərə ala bilsin. `using` iş bitəndə source-un resurslarını təmizləyir.

Öz uzun dövrümüzdə də siqnalı vaxtaşırı yoxlaya bilərik. `ThrowIfCancellationRequested()` istək varsa ləğv xətası verir. Buna əməkdaşlığa əsaslanan ləğv deyilir: kod siqnalı qəbul edir və dayanır.

**Timeout** müəyyən vaxtdan artıq gözləməməkdir. Gözləmənin bitməsi işin özünün mütləq dayandığını göstərmir. Əməliyyatın dayanması lazımdırsa onun cancellation dəstəyi və token-in ona ötürülməsi əhəmiyyətlidir.

İstifadəçi ləğvi ilə vaxt sərhədini birləşdirən linked token də yaratmaq olar: hər iki mənbədən biri ləğv istəyəndə ümumi siqnal verilir.

# Result və Wait niyə problem yarada bilər

`.Result` və `.Wait()` Task bitənə qədər çağıran thread-i bloklaya bilər. UI proqramında bu ekranı dondura bilər. **UI** istifadəçi interfeysidir; onun xüsusi thread-i düymə və ekran yeniləmələrini idarə edir.

Dairəvi gözləmə nümunəsi

```
UI thread: ".Result ile metodun bitmesini gozleyirem."

Metodun davami: "Bitmek ucun UI thread-de islemeliyem."

UI neticeni, netice ise UI thread-i gozleyir.
```

Belə irəliləyə bilməyən qarşılıqlı gözləmə **deadlock** adlanır.

**SynchronizationContext** await-dən sonra davamın hansı mühitdə işləməsini bildirə bilən mexanizmdir. UI-də bu çox vaxt UI thread-idir. `ConfigureAwait(false)` həmin await üçün bu mühitə qayıtmaq tələbini azaltmağa imkan verir. “Mütləq yeni thread aç” və ya “bütün deadlock-ları düzəlt” demək deyil.

ASP.NET Core-da standart klassik request SynchronizationContext yoxdur. Amma bloklama yenə çoxlu thread-i gözləmədə tutub başqa işlər üçün thread çatışmazlığı yarada bilər. Buna **ThreadPool starvation** deyilir.

**İlkin seçim:** asinxron işi mümkün olduqca çağırış zənciri boyunca await et. Gözləmə problemini `.Result` ilə yuxarıya gizlətmə.

# Async void və nəzarətsiz fon işi

`async Task` metodu çağırana tamamlanmanı izləmək üçün Task verir. `async void` isə belə Task vermir. Ona görə onu adi metod kimi await edib tamamlanmasını və xətasını izləmək olmur.

Əsas istisna void imzası tələb edən hadisə işləyicisidir. Məsələn, UI düyməsinin click handler-i. Handler hadisə gələndə çağırılan koddur. Adi iş metodları üçün Task və ya Task nəticəsi olan variant seçilir.

**Fire and forget** işi başladıb sonunu izləməməkdir. HTTP sorğusunda işi belə başlatmaq onun etibarlı bitməsinə zəmanət vermir.

Ömür problemi

```
Sorqu baslayir, DbContext alir.
    |
    +-- gozlenilmeyen fon isi hemin context-i tutur.

Sorqu bitir, scope temizlenir.
Fon isi hele context-den istifade etmek isteyir.
```

DI dərsində scope-un xidmətləri hansı sərhədə qədər saxladığını gördük. Fon işi sorğudan uzun yaşayırsa onun resursları da ayrıca idarə olunmalıdır.

Etibarlı fon emalı üçün tətbiqin idarə etdiyi worker, öz scope-u və iş itməməlidirsə davamlı növbə istifadə oluna bilər. **Worker** arxa planda işi emal edən hissədir. Davamlı növbə isə proqram bağlansa da işi saxlayan mexanizmdir.

# Resurs await bitənə qədər açıq qalmalıdır

Fayldan mətn oxuyuruq. Reader obyektini oxuma tamamlanmamış bağlasaq başladılmış iş artıq bağlanmış resursla qarşılaşa bilər.

C#

```
static async Task<string> ReadAsync(string path)
{
    using var reader = new StreamReader(path);
    return await reader.ReadToEndAsync();
}
```

`StreamReader` fayldan mətn oxuyan obyektdir. `path` faylın yoludur. `ReadToEndAsync` sonadək oxumağı təmsil edən Task verir. Await onun bitməsini gözləyir, sonra metoddan çıxırıq və using reader-i təmizləyir.

Burada await-i düşünmədən çıxarıb hələ tamamlanmamış Task-ı qaytarmaq resursun tez bağlanmasına səbəb ola bilər. **Kodun qısalması həmişə eyni davranışı qorumaq demək deyil.**

İndi **ValueTask** adını da yerinə qoyaq. O, bəzi hallarda hazır nəticəni ayrıca Task yaratmadan təmsil edə bilər. Çox tez-tez dərhal tamamlanan və ölçülmüş xərcin vacib olduğu əməliyyatlarda faydalı ola bilər.

ValueTask həmişə stack-də saxlanır demək deyil. **Stack** çağırışlara aid məlumatın saxlandığı yaddaş sahəsidir. ValueTask başqa obyektin sahəsində də yerləşə bilər; yer onun harada istifadə edilməsindən asılıdır. Ümumi qayda kimi onu bir dəfə await et. Dəfələrlə gözləmək lazımdırsa Task müqaviləsi daha rahatdır. Sadə tətbiq metodlarında başlanğıc seçim çox vaxt Task-dır.

# Eyni məlumatı paralel dəyişəndə nəticə itə bilər

Sayğac 0-dır. İki thread `counter++` edir. Hər iki artım saxlanarsa nəticə 2 olmalıdır. Amma ++ oxu, artır, yaz addımlarından ibarətdir:

| Addım | Thread A | Thread B |
|---|---|---|
| Oxuma | 0 oxuyur | 0 oxuyur |
| Hesablama | 1 hesablayır | 1 hesablayır |
| Yazma | 1 yazır | 1 yazır |

Son nəticə 1 qala bilər. Nəticənin nəzarətsiz icra sırasından asılı olmasına **race condition** deyilir.

`Interlocked.Increment(ref counter)` artımı digər paralel artımlara qarşı bölünməz əməliyyat edir. **Atomic** burada həmin addımın araya qarışdırılmadan yerinə yetirilməsi deməkdir. `ref` dəyişənin özünə istinad ötürülməsini bildirir.

**Lock** müəyyən kod hissəsinə eyni anda girişi məhdudlaşdırır. Adi lock bloku daxilində await istifadə olunmur. Asinxron gözləmə ilə giriş limiti üçün `SemaphoreSlim.WaitAsync` uyğun ola bilər. Semaphore müəyyən sayda icazə saxlayır; iş bitəndə icazə geri verilir.

**Nəticə:** async yazmaq paylaşılan məlumatı avtomatik təhlükəsiz etmir. Bir işi gözləmək ilə eyni məlumata paralel girişi idarə etmək ayrı məsələlərdir.

# Bütün dərsin məntiqini bir işə tətbiq et

Kafe ekranı qiymət və ehtiyat istəyir. Bu iki xarici sorğu bir-birindən asılı deyilsə əvvəl ikisini başlatmaq, sonra WhenAll ilə nəticələri gözləmək olar. Eyni DbContext-i paralel paylaşmaq isə olmaz.

İstifadəçi ekranı bağlayanda ləğv token-i əməliyyatlara ötürülür. Token dayanma istəyidir; hər işi zorla öldürən mexanizm deyil.

Nəticə alınanda UI görünüşü yenilənir. UI tələb olunan kod həmin mühitin thread qaydasına əməl etməlidir. Eyni thread-ə həmişə qayıdacağına və ya hər await-də thread-in dəyişəcəyinə dair ümumi hökm yoxdur.

### Özünü yoxla

**Task ayrıca thread-dirmi?** Xeyr. Əməliyyatın tamamlanmasını və nəticəsini təmsil edir.

**Async CPU hesabını avtomatik sürətləndirirmi?** Xeyr. Gözləməni bloklamadan idarə etməyə imkan verir; CPU işi üçün alqoritm və hesablama resursları ayrıca məsələdir.

**WhenAll-da biri xəta atsa digərləri dayanırmı?** Avtomatik dayanmır. Onların ləğvi ayrıca idarə olunur.

**Bir cümləlik məntiq:** işi başladırıq, tamamlanmanı Task ilə təmsil edirik, lazım olan nəticəni await ilə gözləyirik və işin resurs, xəta, ləğv sərhədlərini ayrıca düzgün saxlayırıq.